



Degree Project in Communication Systems

Second cycle, 30 credits

Single-Query Auto-Segmenting Side-Channel Based Disassembly

MATHIAS DUEDAHL

Single-Query Auto-Segmenting Side-Channel Based Disassembly

MATHIAS DUEDAHL

Master's Programme, Computer Science, 120 credits

Date: August 18, 2025

Supervisors: Jakob Sternby, Sönke Jendral

Examiner: Elena Dubrova

School of Electrical Engineering and Computer Science

Host company: Ericsson

Swedish title: Enmättnings Auto-Segmenterande Sidokanalsbaserad

Instruktionsavkodning

Abstract

Side-channel analysis (SCA) has long been a cornerstone of cryptographic security research, enabling attackers to extract sensitive information by monitoring a device's power consumption or electromagnetic (EM) emissions. This foundation has led researchers to explore side-channel based disassembly (SCBD), which aims to reconstruct a program's instruction sequence directly from side-channel traces. However, practically viable solutions remain elusive due to the trace segmentation bottleneck: existing techniques either rely on averaged traces, classify only isolated instructions, or require custom, device-specific logic to segment continuous traces.

This segmentation challenge has been extensively studied in automatic speech recognition (ASR), where segmenting continuous audio streams into discrete phonemes or words is critical. Despite striking similarities between SCA and ASR challenges, cross-domain approaches remain unexplored in SCBD. We investigate whether modern ASR models can be adapted to solve the segmentation problem in SCBD.

We developed a comprehensive experimental framework using a custom RISC-V (RV) instruction sequence generator (ISG) and implemented a RV32 softcore on a field-programmable gate array (FPGA) to generate training data. We adapted the wav2vec 2.0 architecture, pretraining on unlabeled power trace data followed by fine-tuning with connectionist temporal classification (CTC) loss on labeled instruction sequences. This approach automatically handles segmentation while learning instruction-specific power signatures.

Our method achieved an accuracy of 76.61% for sequences containing five different RV32I instructions, with segmentation error rates (SERs) as low as 6.86%. These results demonstrate that ASR models can effectively solve the temporal alignment challenge in SCBD, with remaining errors concentrated in instruction discrimination rather than segmentation. Statistical validation confirms significant improvements over baseline approaches ($p < 0.001$). Our findings establish ASR as a viable foundation for SCBD and demonstrate the potential of cross-domain approaches for practical disassembly techniques.

Keywords

Side-Channel Analysis, Side-Channel Based Disassembly, Automatic Speech Recognition, wav2vec 2.0, Connectionist Temporal Classification

Sammanfattning

Sidokanalsanalys (SCA) har länge varit en hörnsten inom kryptografisk säkerhetsforskning, och möjliggör för angripare att extrahera känslig information genom att övervaka en enhets strömförbrukning eller elektromagnetiska (EM) läckage. Denna grund har lett forskare att utforska sidokanalbaserad instruktionsavkodning (SCBD), som syftar till att rekonstruera ett programs instruktionssekvens direkt från sidokanalsmätningar. Dock förblir praktiskt användbara lösningar orealistiska på grund av flaskhalsen för mätningsegmentering: befintliga tekniker förlitar sig antingen på medelvärdesbildning, klassificerar endast isolerade instruktioner, eller kräver anpassad, enhetsspecifik logik för att segmentera kontinuerliga mätningar.

Denna segmenteringsutmaning har studerats utförligt inom automatisk taligenkänning (ASR), där segmentering av kontinuerliga ljudströmmar till diskreta fonem eller ord är kritisk. Trots slående likheter i problembeskrivningen för SCA och ASR har ansatser från taligenkänningsområdet förblivit outforskade inom SCBD. Vi undersöker huruvida moderna ASR-modeller kan anpassas för att lösa segmenteringsproblemet inom SCBD.

Vi har utvecklat ett omfattande experimentellt ramverk med hjälp av en anpassad RISC-V (RV) instruktionssekvensgenererare (ISG) och implementerade en RV32 softcore på en fältprogrammerbar grindmatris (FPGA) för att generera träningsdata. Vi anpassade wav2vec 2.0-arkitekturen, förtränade på omärkta strömspårsdata följt av finjustering med konnektionistisk temporal klassificering (CTC)-förlust på märkta instruktionssekvenser. Denna ansats hanterar automatiskt segmentering samtidigt som den lär sig instruktionsspecifika strömsignaturer.

Vår metod uppnådde en noggrannhet på 76.61% för sekvenser innehållande fem olika RV32I-instruktioner, med segmenteringsfelsfrekvenser (SER) så låga som 6,86%. Dessa resultat visar att ASR-modeller effektivt kan lösa den temporala anpassningsutmaningen inom SCBD, med kvarvarande fel koncentrerade till instruktionsdiskriminering snarare än segmentering. Statistisk validering bekräftar betydande förbättringar jämfört med en naiv jämförelsemetod ($p < 0,001$). Våra fynd etablerar ASR som en möjlig utgångspunkt för SCBD och demonstrerar potentialen av att dra lärdom från andra domäner för att göra framsteg inom metoder för instruktionsavkodning.

Nyckelord

Sidokanalsanalys, Sidokanalsbaserad Instruktionsavkodning, Automatisk Taligenkänning, wav2vec 2.0, Konnektionistisk Temporal Klassificering

Acknowledgments

I would like to extend my sincerest gratitude to my brilliant colleagues at Ericsson's Hardware Security group. To Niklas Lindskog for opening the door for me although I was half the world away, and for paving the way for me to join his team in Lund. To Håkan Englund for tirelessly fighting through the depths of FPGA hell with me, and always reminding us of the apple cake. And to my supervisor, Jakob Sternby, for keeping me company over the summer, with many interesting talks and invaluable sparring.

To all of you, thank you for making me feel welcome in your team. Without your support and generously shared expertise, this project could not have become reality. I could not have asked for a finer group of people to work with for my final sprint before the finish line.

I would also like to thank my KTH supervisor Sönke Jendral, and professor Elena Dubrova for allowing me the freedom to pursue this project in the way I envisioned.

And lastly, I would like to thank my family, whom I can always count on for unwavering support and encouragement.

Stockholm, August 2025

Mathias Duedahl

Contents

1	Introduction	1
1.1	Background	1
1.1.1	The Segmentation Problem	2
1.1.2	The Deep Learning Gap	2
1.1.3	The Motivation	3
1.2	Problem	4
1.3	Purpose	4
1.4	Goals	5
1.5	Research Methodology	5
1.6	Delimitations	6
2	Background	9
2.1	Hardware Foundations for SCA	9
2.1.1	Assembly Instruction Anatomy	9
2.1.2	Processor Architecture	11
2.1.3	Instruction Cycle	13
2.1.4	Side-Channel Leakage	14
2.2	Automatic Speech Recognition	16
2.2.1	Connectionist Temporal Classification	16
2.2.1.1	Alignment	16
2.2.1.2	Training	17
2.2.1.3	Inference	18
2.2.2	Transformers	19
2.2.2.1	Self-Attention Head	19
2.2.2.2	Multi-headed Attention	20
2.2.2.3	Positional Encoding	22
2.2.2.4	Architecture	22
2.2.3	wav2vec 2.0	24
2.2.3.1	Encoder	25

2.2.3.2	Quantization Module	25
2.2.3.3	Context Network	26
2.2.3.4	Training	26
2.3	Related work	28
2.3.1	Side-Channel Based Disassembly	28
2.3.2	Neural Networks in Side-Channel Analysis	28
2.3.3	Acoustic Side-Channels	29
2.3.4	Literature Gap Analysis	29
3	Data Collection Pipeline	31
3.1	Target System Architecture	31
3.1.1	Hardware Setup	31
3.1.2	Softcore Implementation	32
3.1.3	System Interface	34
3.2	Data Generation Pipeline	34
3.2.1	Instruction Sequence Generation	35
3.2.2	Compilation Toolchain	36
3.2.3	Data Generation	37
3.2.3.1	Hardware Debugging	37
3.2.3.2	Simulation	37
3.3	Data Validation	38
3.3.1	Label Validation	38
3.3.2	Trace Validation	38
3.4	Data Preprocessing	40
3.5	Data Inspection	40
3.5.1	Intra-Group Similarity	40
3.5.2	Inter-Group Distinguishability	41
3.5.3	Instruction Set Downscoping	42
4	Model Development	43
4.1	Model Adaptation	43
4.1.1	Tokenization Strategy	44
4.2	Dataset Generation	44
4.3	Hyperparameter Optimization Framework	45
4.4	Training Strategy	47
4.5	Evaluation Metrics	48
4.5.1	Word Error Rate	48
4.5.2	Segmentation Error Rate	49
4.5.3	ASR Adapted McNemar	49

5	Results and Analysis	51
5.1	Hyperparameter Tuning	51
5.1.1	Configuration Selection	54
5.1.2	Finetuning Optimization	55
5.2	Training Metrics	56
5.2.1	Pretraining	56
5.2.2	Finetuning	58
5.3	Evaluation	58
5.3.1	Statistical Validation	59
5.3.2	Segmentation Error Analysis	59
5.3.3	Error Pattern Analysis	60
6	Conclusions	63
6.1	Limitations	63
6.2	Future Work	64
6.2.1	Immediate Extensions	64
6.2.2	Advanced Architectures	64
6.2.3	Fundamental Research Questions	64
	References	67
A	Trace Visualization	75
B	ASR Adapted McNemar Derivation	78
B.1	Standard McNemar Test Foundation	78
B.2	Adaptation for Transcription Models	78
B.3	Constructing Discordant Pairs	78
B.4	Assignment of Insertions	79
B.5	Final Test Statistic	79
B.6	Independence Assumption	79
C	Hyperparameter Tuning Visuals	80
D	Word Confusion Matrices	82

List of Figures

2.1	RV base instruction formats	10
2.2	RV R-type machine instruction	11
2.3	Complete single-cycle processor diagram	12
2.4	Ridge plot of power traces from <code>add</code> and <code>lw</code> instructions . . .	15
2.5	CTC repeated token collapse	16
2.6	CTC collapse with blank tokens	17
2.7	CTC alignment merging	18
2.8	Attention head	20
2.9	Multi-Head attention block	21
2.10	Transformer architecture	23
2.11	wav2vec 2.0 architecture	25
3.1	Experimental setup	32
3.2	Ibex demo system diagram.	33
3.3	Ibex Core pipeline diagram.	33
3.4	Overlaid power traces, head- and tail-aligned.	39
3.5	Ridge plot of power traces for RV32I I-Type instructions . . .	41
3.6	Ridge plot of the chosen subset of five mutually distinguish- able RV32I instructions.	41
4.1	Overview of hyperparameter tuning strategy	48
5.1	Slice plot of hyperparameters for pretrain Optuna trials	51
5.2	Pretraining Optuna parameter importance	53
5.3	Pretraining loss for multiple Optuna trials	54
5.4	Finetuning initialization metrics	55
5.5	Metrics for the full pretraining of models A–D	57
5.6	Metrics for the full finetuning of models A–D	58
5.7	Word confusion matrix for Finetune Full C	61

A.1	Ridge plot of power traces for RV32I B-Type instructions. . . .	75
A.2	Ridge plot of power traces for RV32I Load/Store-Type instructions.	75
A.3	Ridge plot of power traces for RV32I R-Type instructions. . . .	76
A.4	Ridge plot of power traces for RV32I Non-Standard instructions.	76
A.5	Ridge plot of power traces for all RV32I instructions.	77
C.1	Metrics for the pretrain-refined- stage	80
C.2	Finetuning stage Optuna parameter importances.	80
C.3	Finetuning stage Optuna parameter importances.	81
D.1	Word confusion matrices for each of the Finetune Full A-D models	82

List of Tables

2.1	Comparison of related work on SCBD	29
4.1	Dataset metadata for rvi_mdb variants	45
4.2	Hyperparameter search space for pretraining optimization. Square brackets denote a range, and curly braces denote a set. .	46
4.3	Hyperparameter search space for finetuning optimization. Square brackets denote a range, and curly braces denote a set. .	46
5.1	Base model configurations	55
5.2	Best Optuna hyperparameter configuration for branches A–D .	56
5.3	WER results for branches A–D across all splits.	58
5.4	ASR Adapted McNemar test results	59
5.5	SER results for branches A–D across all splits.	60
B.1	Contingency table for correctness of two classifiers, H_0 and H_a . .	78

Listings

- 3.1 C file structure for each generated software. Where buffer register is a register that will be avoided by the ISG, and the ISG sequence is substituted in place of the braces in the volatile inline assembly block. 35
- 3.2 Simplified section of the main.c file showing interplay between the simpleserial SDK and our execute_cw() function . 36

List of acronyms and abbreviations

ALU	arithmetic logic unit
ASR	automatic speech recognition
CNN	convolutional neural network
CPU	central processing unit
CTC	connectionist temporal classification
DUT	device under test
EM	electromagnetic
EX	execution cycle
FPGA	field-programmable gate array
ID	instruction decode cycle
IF	instruction fetch cycle
ISA	instruction set architecture
ISG	instruction sequence generator
MEM	memory access cycle
MLP	multilayer perceptron
PC	program counter
RISC	reduced instruction set computer
RNN	recurrent neural network
RV	RISC-V
SCA	side-channel analysis
SCBD	side-channel based disassembly
SER	segmentation error rate
SoC	system on a chip
WB	write-back cycle
WER	word error rate

Chapter 1

Introduction

In this thesis we investigate whether automatic speech recognition (ASR) models can transfer to the domain of side-channel based disassembly (SCBD). Specifically, we explore whether the speech-to-text paradigm of ASR models is suited to transcribing assembly instruction sequences from their associated side-channel power traces.

1.1 Background

Modern processors, despite extensive security measures, inevitably leak information through power consumption and electromagnetic (EM) radiation that corresponds to their internal circuitry usage. The field of side-channel analysis (SCA) exploits this physical reality, using statistical modeling to infer information about processor operations purely from observable side-channel emissions such as power consumption. This enables powerful capabilities including cryptographic key extraction, malware detection, and software fingerprinting [1–5].

SCBD represents the most granular form of SCA, attempting to recover the sequence of instructions executing on a processor at the assembly or machine code level. A functioning SCBD system would enable all the traditional use cases of SCA while opening new possibilities including reverse engineering, debugging through side-channel information, and potentially provably adversarially robust security systems. This fine-grained analysis capability is particularly valuable for malware detection, where existing side-channel based systems are susceptible to adversarial attacks [6]. By disassembling instructions at the bit level, SCBD could enable security

guarantees that are difficult to achieve at coarser granularities, since the granularity is finer than what attackers can feasibly control - they cannot modify the behavior of single instructions without access to microcode-level modifications.

1.1.1 The Segmentation Problem

Current SCBD approaches face significant technical challenges that limit their practical deployment. Most existing methods require extensive manual effort to build instruction templates or perform trace segmentation - the process of identifying where individual instructions begin and end within continuous power traces. This segmentation problem is particularly pronounced because:

1. Instructions vary in execution time and power consumption patterns
2. Modern processors employ out-of-order execution, speculation, and parallel processing
3. Power traces contain noise and artifacts from system-level activity

These challenges have led to SCBD remaining largely in the academic domain, with most practical SCA applications operating at coarser granularities where instruction recovery is unnecessary.

1.1.2 The Deep Learning Gap

Interestingly, while the broader field of SCA has embraced deep learning - enabling breakthroughs in cryptanalysis, such as improved side-channel attacks on AES implementations [7] - SCBD has seen limited adoption of modern neural architectures.

Most SCBD research still relies on traditional machine learning techniques, as will be discussed in section 2.3. This is surprising given that SCBD represents the most complex signal analysis task in the SCA domain and should theoretically benefit most from advanced deep learning techniques.

The core challenges of SCBD - extracting discrete symbolic sequences from continuous, noisy temporal signals - share striking parallels with ASR. Both domains must solve the fundamental problem of segmentation and alignment: determining where discrete units (instructions or words) begin and end within continuous signals (power traces or audio). Modern ASR systems have solved remarkably similar challenges:

Temporal Segmentation: ASR models excel at identifying word boundaries in continuous speech, just as SCBD requires identifying instruction boundaries in power traces.

Variable Length Units: Words vary in duration and acoustic properties, just as instructions vary in execution time and power signatures.

Parallel Processing: Modern ASR handles overlapping speech from multiple speakers, analogous to how SCBD must handle parallel execution units and out-of-order processing.

Domain Adaptation: ASR systems adapt across languages, accents, and speaking styles, while SCBD must generalize across different processor architectures and instruction set architectures (ISAs).

These parallels suggest that the sophisticated attention mechanisms, sequence modeling capabilities, and end-to-end learning approaches that have revolutionized speech recognition may be directly applicable to SCBD.

1.1.3 The Motivation

The convergence of these factors - the practical importance of SCBD, the limitations of current approaches, and the untapped potential of modern deep learning - motivates this research. We hypothesize that ASR models, with their proven ability to extract discrete sequences from continuous temporal signals, can overcome the traditional limitations of SCBD systems.

If successful, this approach could enable automated, template-free disassembly that scales beyond current manual methods, potentially opening new frontiers in security analysis, reverse engineering, and adversarially robust system design. The fine-grained nature of instruction-level analysis could provide security guarantees that are difficult to achieve at coarser granularities, since individual instruction execution patterns are below the level of control available to most attackers.

This thesis explores whether this interdisciplinary approach can bridge the gap between the sophisticated signal processing capabilities of modern ASR and the challenging problem of SCBD, potentially catalyzing a new generation of automated reverse engineering tools.

1.2 Problem

Researchers have successfully built a multitude of SCBD models with extraordinarily high accuracies in recent years [8–10]. However, all of them come with significant limitations, such as requiring signal averaging over hundreds of repetitions, manual segmentation, downclocked and/or single-core hardware, etc. These constraints prevent the models from being applicable in real-world settings.

In this work, we aim to test the feasibility of adapting models from the speech-to-text domain to SCBD in order to circumvent the segmentation problem without requiring averaging of multiple signals. Specifically, we investigate whether the wav2vec 2.0 model trained with connectionist temporal classification (CTC) loss is capable of automatically segmenting and disassembling instruction sequences from single-query power traces.

1.3 Purpose

This work aims to bridge the gap between ASR and SCA to advance the capabilities of side-channel attacks. SCBD represents one of the most challenging problems in SCA, making it an ideal testbed for novel approaches. Success in this domain would likely have cascading effects across the broader field of SCA. Furthermore, the success of foundation models in ASR - which can recognize speech across diverse speakers, accents, and recording conditions - suggests that universal models capable of classifying instruction sequences across different hardware platforms may also be within reach for SCBD.

The potential applications of this research are significant. Ericsson has conducted extensive research in side-channel-based malware detection and cryptographic side-channel attacks, where more fine-grained processor insights would prove valuable. More broadly, this work could enable practical applications in any scenario requiring air-gapped monitoring, including malware detection, debugging, and reverse engineering of isolated systems.

However, these capabilities raise important ethical considerations, particularly regarding intellectual property protection. If power or EM side-channel traces can accurately reveal full instruction sequences, fundamental questions emerge about data and IP confidentiality. Hardware manufacturers selling devices with proprietary software face increased vulnerability to reverse

engineering of their intellectual property. That said, mapping instruction sequences to their originating software remains a significant obstacle, so automated reverse engineering is not yet an immediate concern even with perfect SCBD.

Should the reverse engineering challenge be overcome, the technology could serve critical industrial applications. The banking sector, for instance, struggles with technical debt and reliance on proprietary legacy systems from vendors who may no longer exist [11]. In such cases, air-gapped reverse engineering could enable the migration of legacy systems and code to modern equivalents, addressing a pressing industry need.

The dual nature of this research - offering both security insights and potential vulnerabilities - underscores the importance of developing these capabilities responsibly while understanding their full implications. By conducting this research in an open, academic setting, hardware vendors are afforded the opportunity to develop and implement appropriate mitigations before such techniques could be exploited maliciously.

1.4 Goals

The goal of this project is to assess the viability of adapting ASR models to the domain of SCBD. This has been divided into the following two sub-goals:

1. Adaptation of the wav2vec 2.0 model with CTC loss to work with power trace data and assembly instructions as tokens.
2. Hyperparameter tuning and training the model to achieve the best possible accuracy for SCBD.

1.5 Research Methodology

We answer the research question by way of the following steps:

1. **Literature Study:** Conduct a comprehensive review of three key areas: ASR models and architectures (particularly wav2vec 2.0 and CTC-based approaches), existing SCA and SCBD research methodologies, and RISC-V ISA to establish domain knowledge for the target platform.
2. **Experimental Setup:** Establish measurement and debugging capabilities for the device under test (DUT), including power trace acquisition

hardware and data collection protocols to ensure reproducible side-channel measurements.

3. **Data Generation Framework:** Develop a systematic framework for generating labeled training data, including: automated generation of diverse RISC-V assembly programs, synchronized collection of power traces, and creation of ground-truth instruction labels for supervised training.
4. **Data Exploration and Preprocessing:** Analyze the collected side-channel data to identify potential issues such as signal quality variations, noise characteristics, instruction-dependent power signatures, and temporal alignment challenges. Implement appropriate preprocessing.
5. **Model Architecture Adaptation:** Modify wav2vec 2.0's architecture to handle power trace inputs instead of audio signals, including: input layer modifications for power trace characteristics, tokenization strategy for assembly instructions, and adaptation of the CTC loss function for instruction sequence prediction.
6. **Hyperparameter Optimization:** Systematically tune hyperparameters for both pretraining and fine-tuning stages, adapting parameters from the audio domain to the side-channel domain.
7. **Model Training and Validation:** Train the adapted model using appropriate train/validation/test splits.
8. **Performance Evaluation:** Assess model performance using word-error-rate, comparing against results from the literature, and significance testing against its naive majority-class counterpart.
9. **Reproducibility and Documentation:** Document all experimental procedures, hyperparameters, and results to ensure reproducibility, and release code and datasets where appropriate to support future research.

1.6 Delimitations

The motivation for this project opens the door to many interesting challenges. In order to limit the project to a reasonable scope, we will not:

Assess other ASR models beyond wav2vec 2.0 with CTC: Due to time and computational resource constraints, this study focuses on a single model

architecture to enable thorough investigation of the domain adaptation process.

Evaluate cross-platform generalization: The model will be assessed only on the target RISC-V platform due to time constraints and limited access to diverse hardware platforms and ISA specifications.

Investigate hardware-specific optimizations: While this would be valuable for future research, time constraints necessitate focusing on the core adaptation challenge, and leave the question of whether ASR models are capable of handling hardware optimization to other researchers.

Attempt bit-level or register-level decoding: Due to time constraints, the study focuses on instruction-level classification rather than extending to finer-grained analysis of individual bits or register states.

Address real-world deployment considerations: Practical aspects such as noise robustness in uncontrolled environments, computational efficiency for real-time analysis, and countermeasure resilience are beyond the scope of this foundational study.

Chapter 2

Background

This chapter establishes the foundational knowledge necessary to understand how SCA can reveal executing instructions and how machine learning techniques can automate this process. We begin by examining the hardware foundations that create observable side-channel leakage, covering processor architecture and assembly language structure. Next, we explore machine learning concepts from automatic speech recognition that prove applicable to the instruction segmentation problem.

Finally, we review related work in SCA and machine learning approaches to contextualize our contribution.

2.1 Hardware Foundations for SCA

Side-channel leakage originates from the physical reality that computation generates observable changes in power consumption and EM emissions. These physical manifestations are directly caused by the data-dependent switching activity within the device's circuitry. To understand how these phenomena reveal executing instructions, we build understanding from the ground up: first examining assembly language - the fundamental language processors understand - then the hardware machinery that processes these instructions.

2.1.1 Assembly Instruction Anatomy

At the lowest level, processors execute binary machine code - sequences of ones and zeros that directly control hardware operations. Assembly language provides a human-readable representation of these machine instructions, with

assembly instructions corresponding directly to machine instructions*, though the specific encoding format varies between different ISAs.

Understanding instruction formats provides the foundation for later examining how different instruction types give rise to physically observable differences during execution. Each instruction contains encoded information about what operation to perform and which data to operate on, and these structural differences will prove relevant when we examine the physical implementation of instruction processing in Sections 2.1.2 and 2.1.3.

This thesis focuses on RISC-V (RV), a modern ISA that is freely available as an open standard, allowing anyone to access the complete specifications. All RV instructions are 32 bits long and follow four primary formats (R/I/S/U), as shown in Figure 2.1. Additional formats exist beyond these four, but all share a common feature: the opcode field always occupies the seven least significant bits [12].

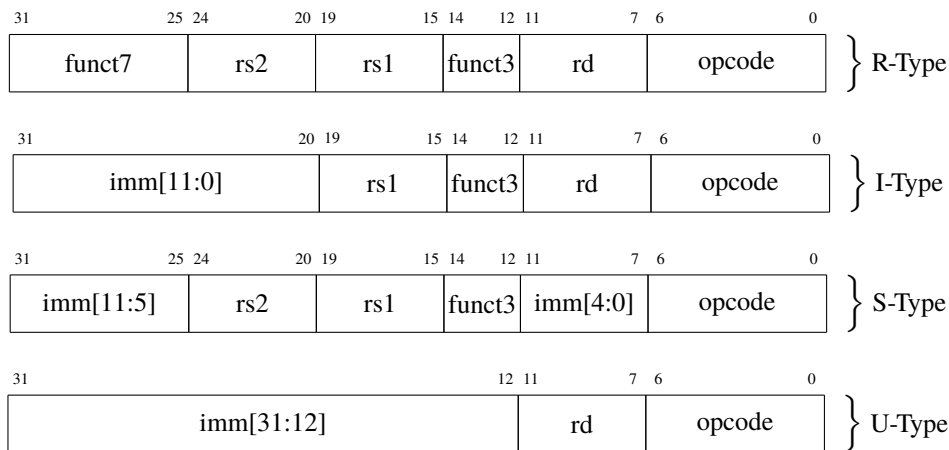


Figure 2.1: RV base instruction formats [12, ch. 2.2]

Each instruction format encodes different types of operations and operand access patterns. The key fields serve distinct roles in instruction execution, these roles are as follows [13, ch. 2.5]:

opcode Specifies the primary operation and instruction format

rd Identifies the destination register for storing results

*If we ignore pseudo-instructions

rs1, rs2 Specify source registers containing input operands

funct3, funct7 Provide additional operation encoding

imm Contains immediate values embedded directly in the instruction

To illustrate these concepts, consider the concrete example shown in Figure 2.2, which shows the machine instruction corresponding to `add x1, x2, x3`.

The opcode field classifies it as an R-type instruction, one must now examine `funct3` and `funct7` to determine that this is in fact an `add` instruction. The operands can then be decoded by inspecting the `rd`, `rs1` and `rs2` fields, which contain the values 1, 2 and 3 respectively, corresponding to registers `x1`, `x2` and `x3`.

31	25 24	20 19	15 14	12 11	7 6	0
funct7	rs2	rs1	funct3	rd	opcode	
0000000	00011	00010	000	00001	0110011	

Figure 2.2: RV R-type machine instruction corresponding to the assembly instruction `add x1, x2, x3` [13, ch. 2.5].

This encoding demonstrates the mapping between assembly instructions and the specific bit patterns that the processor’s hardware must interpret and execute. The question naturally arises: how does the processor actually perform this interpretation and execution?

2.1.2 Processor Architecture

Having examined how instructions are encoded, we now turn to the hardware components that decode and execute them. While modern processors are complex systems with many specialized units, certain core components are fundamental to instruction processing and directly relevant to SCA. Figure 2.3 illustrates these key functional units and their interconnections in a complete processor.

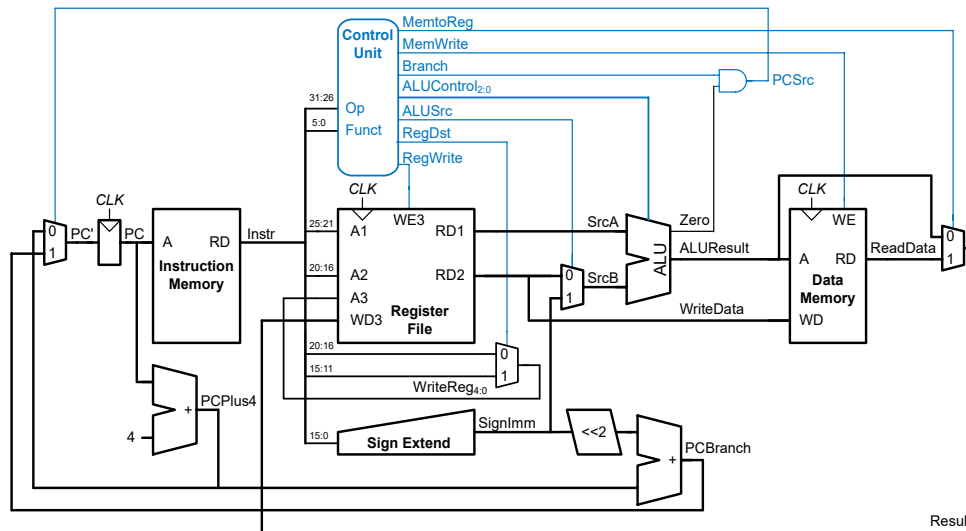


Figure 2.3: Complete single-cycle processor diagram, displaying the major functional units and their mutual connections [14, ch. 7.3]

The following components are particularly important for understanding side-channel leakage, as each contributes distinct characteristics to the processor's power consumption and EM emissions:

Arithmetic logic unit (ALU)

Performs arithmetic and logical operations such as addition, subtraction, AND, OR, and comparisons. Different operations may utilize separate specialized circuits (adders, shifters, multipliers, dividers), with implementations varying in whether these are integrated within the ALU or implemented as dedicated functional units [14, ch. 5.7].

Register File

Stores the processor's working registers as an organized collection with multiple access ports. The register file in Figure 2.3 provides two read ports and one write port, enabling simultaneous access operations [14, ch. 5.5.5].

Control Unit

Decodes instructions and generates control signals based on opcode and function fields [14, ch. 7.3.2]. These control signals coordinate the behavior of other functional units, as indicated by the blue control lines in Figure 2.3.

Data Memory

Handles data storage and retrieval operations, reading or writing based on control signals from the control unit, particularly the write enable (WE) signal [14, ch. 7.1.2].

Program counter (PC)

Maintains the address of the current instruction. While typically incremented sequentially, the PC can be modified for branches and jumps as directed by the control unit [14, ch. 6.4.2].

These functional units work together to process each instruction, but their coordination reveals an important characteristic: executing a single instruction requires engaging multiple components sequentially.

Instructions must pass through several sequential phases that create temporal patterns crucial for SCA.

2.1.3 Instruction Cycle

The sequential processing described in the previous section occurs through a structured instruction cycle. While early processors executed instructions one at a time, most modern central processing units (CPUs) use pipelining, partitioning instruction execution into separate stages that can operate simultaneously. This allows for higher instruction throughput by concurrently processing multiple instructions, each at different pipeline stages.

A classic reduced instruction set computer (RISC) pipeline consists of the following 5 stages [14, ch. 7.5] [15, ch. C.3]:

Instruction fetch cycle (IF)

The processor retrieves the instruction from instruction memory using the PC.

Instruction decode cycle (ID)

The control unit decodes the instruction and the register file provides operands.

Execution cycle (EX)

The ALU performs the specified operation on the operands from the ID stage.

Memory access cycle (MEM)

For load/store instructions, the data memory unit accesses memory at the address computed in the EX stage.

Write-back cycle (WB)

The result is written back to the register file.

Each pipeline stage engages different functional units from Section 2.1.2, creating distinct temporal patterns in power consumption and EM emissions due to differences in the utilized circuitry. With this foundation, we have covered the hardware concepts required to understand the intricacies of side-channel leakage.

2.1.4 Side-Channel Leakage

The CPU architecture and instruction processing concepts described in the previous sections, form the foundation for understanding how side-channel leakage enables instruction-level analysis of executing programs. The physical implementation of the hardware components discussed in Section 2.1.2 inadvertently reveals information about the computations being performed, creating opportunities for SCBD.

Every digital operation we've discussed - instruction decoding, ALU operations, register file access, memory transactions - has a physical manifestation in the CPU's circuitry. As described in Section 2.1.3, when instructions pass through the five pipeline stages, each stage engages different functional units with distinct electrical characteristics.

When the control unit decodes different instruction types, it activates different circuit paths. An R-type instruction like the `add x1, x2, x3` example from Figure 2.2 requires the ALU to perform arithmetic operations, while an I-type load instruction engages the data memory unit. When the ALU performs multiplication versus addition, it utilizes different functional units with distinct power requirements. When the register file accesses different registers, it will correspondingly activate different circuitry.

These physical differences give rise to measurable variations in two physical observables:

Power Consumption

Different instructions follow different paths through the CPU circuitry, with differing behavior in the functional units, which causes a corresponding difference in the net current drawn to sustain the CPU's operation.

Additionally, the power consumption of digital circuits depends on switching activity - when transistors change state from 0 to 1 or vice versa. This switching activity is often correlated with the Hamming weight (number of 1-bits) of the data being processed, making power consumption data-dependent as well as instruction-dependent [16].

Electromagnetic Radiation

As current flows through circuits, it emits EM radiation. Thus, different current patterns along different circuit paths produce distinct EM radiation signatures.

Consider how the instruction formats from Figure 2.1 translate to observable differences: suppose a processor alternates between executing `add` (R-type) and `lw` (I-type load). These instructions not only have different bit patterns in their opcode and function fields, but they also engage fundamentally different execution paths through the pipeline stages. The `ADD` instruction primarily exercises the ALU during the EX stage, while the load instruction requires data memory access during the MEM stage. This difference manifests as distinct power consumption patterns that can be measured and analyzed to identify which instruction type is being executed, Figure 2.4 demonstrates the difference in power consumption as measured using our own lab setup.

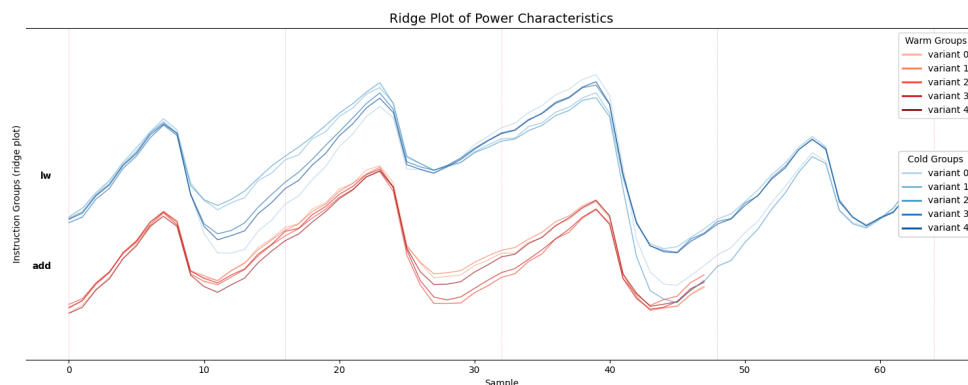


Figure 2.4: Ridge plot of power traces from `add` and `lw` instructions, each overlaid with variants of the same instruction using different registers and immediates. Each clock cycle is demarcated by vertical lines.

More subtly, even the same instruction operating on different data can produce distinguishable signatures. The power consumed by `add x1, x2, x3`

may vary depending on the bit patterns in registers x_2 and x_3 due to the Hamming weight of the operands and the switching activity in the ALU circuits.

2.2 Automatic Speech Recognition

2.2.1 Connectionist Temporal Classification

Recurrent neural networks (RNNs) have long been a go-to model for predicting label sequences from noisy input data. However, RNNs face two significant shortcomings:

1. They require the input data to be pre-segmented. - For instance, in an audio transcription task, it would need knowledge of which segment of the input sequence corresponds to which word, and additionally.
2. Post processing is required to remove redundant labels in the output.

In 2006 Graves et al. introduced the CTC technique, which resolves these shortcomings [17].

2.2.1.1 Alignment

Consider an RNN that produces the most likely label at each time-step. Let $\mathcal{X}$ be the set of all waveforms that correspond to the same label, $Y = [c, a, t]$. Depending on pronunciation, the RNN may (correctly) decode these different waveforms to *alignments* looking like "ccaaaat", "caatttt" etc.

To handle this redundancy of repeated labels, CTC simply collapses repeated tokens, see Figure 2.5 [18].

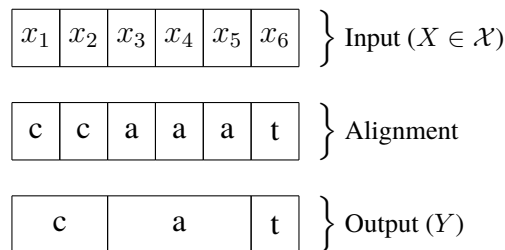


Figure 2.5: CTC repeated token collapse

This fails to account for cases where repeated tokens are intended, such as for words like "hello". To account for this, a *blank* token, $\emptyset$, is introduced.

Thus, allowing the model to assign a placeholder token for periods of silence, or when otherwise deemed necessary as a separator. This allows the collapsing post processing to handle repeated entries by first merging repeated characters, and subsequently removing the blank tokens, see Figure 2.6.

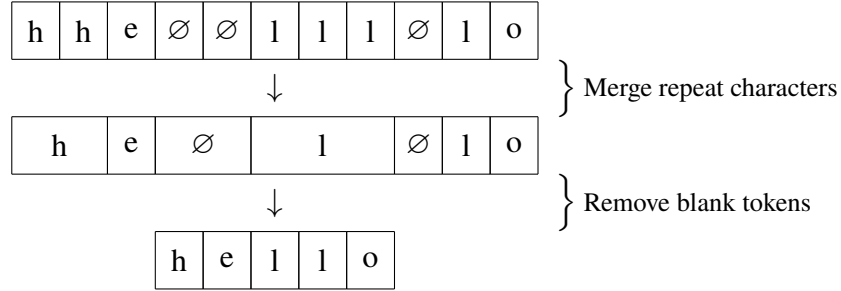


Figure 2.6: CTC collapse with blank tokens

This means, that after preprocessing, all alignments now should correspond to the label Y .

Now that we have introduced the concept of alignments, let us define them more formally: Denote by $\mathcal{X}$ the input space, and by $\mathcal{X}^*$ all sequences over the input space. Similarly, let $\mathcal{Y}$ denote the output (label) space, and $\mathcal{Y}^*$ all sequences over the output space. We extend the output space with the blank symbol introduced previously $\bar{\mathcal{Y}} = \mathcal{Y} \cup \emptyset$.

We can now define the earlier *collapsing* procedure as a many-to-one map $\mathcal{A} : \bar{\mathcal{Y}}_T^* \mapsto \mathcal{Y}_{\leq T}^*$, where the subscript denotes the sequence length that the alignment must map to a sequence of equal or shorter length. The inverse map, $\mathcal{A}^{-1}(Y)$, then denotes the set of all alignments for some label sequence, $Y \in \mathcal{Y}_{\leq T}^*$.

2.2.1.2 Training

For CTC training, the objective for an $(X, Y) \in (\mathcal{X}^*, \mathcal{Y}^*)$ pair is given by $\Pr(Y|X)$, we can obtain this probability by marginalizing over all possible alignments for (X, Y) and further rewrite by assuming network outputs to be conditionally independent in time:

$$\Pr_{CTC}(Y|X) = \sum_{A \in \mathcal{A}^{-1}(Y)} \Pr(A|X) = \sum_{A \in \mathcal{A}^{-1}(Y)} \prod_{(k,t) \in A} \Pr(k|X, t), \quad (2.1)$$

where alignment A is decomposed into T pairs of corresponding label and timestep, (k, t) .

These objective scores can't easily be calculated naively in the manner suggested by Equation 2.1. Assume Y has no repeat symbols, and that $\text{len}(Y) = U$, $\text{len}(X) = T$, then $|\mathcal{A}^{-1}(Y)| \geq \binom{T}{U}$. So we face the issue that $|\mathcal{A}^{-1}(Y)|$ may explode combinatorically.

Fortunately, by realizing that alignments reaching the same output at the same step can be merged, we can use dynamic programming to move forward, see Figure 2.7.

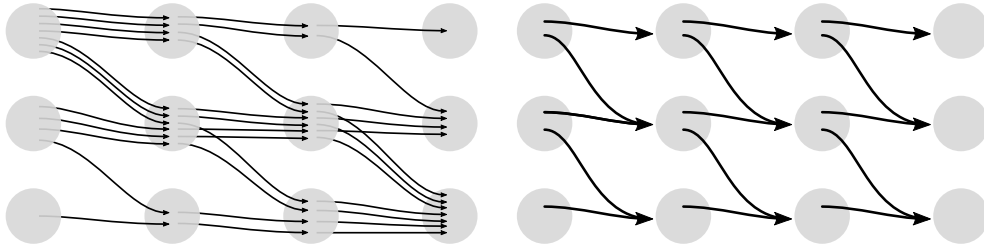


Figure 2.7: CTC alignment merging with dynamic programming [18]

These probabilities can be calculated efficiently using a modified version of the well known forward-backward algorithm to avoid calculating probabilities for each alignment individually, but rather across all alignments simultaneously [17]. Note, the specific expression for how to calculate the probabilities are in terms of the forward backward algorithm, and will not be discussed further here.

The CTC objective in Equation 2.1 can then be used to perform maximum likelihood training, i.e. simultaneously maximizing the probabilities of attaining correct classifications over the training set, or equivalently, minimizing the negative log-likelihood, seen here as the loss function $\mathcal{L}_{CTC}$

$$\mathcal{L}_{CTC} = -\log \Pr_{CTC}(Y|X).$$

2.2.1.3 Inference

At inference time we would like to find the most likely label sequence given our input sequence, i.e. we wish to solve

$$Y^* = \arg \max_Y \Pr(Y|X).$$

However, we currently have no algorithms that make this optimization problem tractable. So we shall settle for a heuristic. Several decent heuristics exist, such as *best path decoding*, *prefix search decoding*, and *beam search*, each with their own virtues and drawbacks. However, upon further development of CTCs successor, Graves et al. deemed Beam search the superior heuristic [19].

2.2.2 Transformers

While CTC focuses on sequence transduction with explicit alignment mechanisms, transformers represent a fundamentally different approach to sequence modeling. Introduced by Vaswani et al. in 2017 [20], transformers rely entirely on attention mechanisms rather than recurrence or explicit alignment.

What makes transformers stand out is their ability to process all tokens in parallel while capturing global dependencies across the entire sequence. This enables the embedding of "cat" in "feed my cat" and "get a cat scan" to accurately capture the contextual meaning without ambiguity.

2.2.2.1 Self-Attention Head

The attention mechanism is what enables transformers to augment embeddings with contextual meaning. After the word embedding step, a word will have a specific embedding regardless of the contextual meaning. To enrich the embedding with contextual meaning, Vaswani et al. take inspiration from retrieval systems and learn three projections onto different subspaces:

1. **Query (Q):** Represents what information the token needs from the context
2. **Key (K):** Represents what information a token can provide to others
3. **Value (V):** Contains the actual content information to be aggregated

These projections are created by multiplying the input embeddings with learned weight matrices (W^Q, W^K, W^V). The model then calculates attention scores by taking the dot product between queries and keys, which determines how much each token should attend to every other token by way of directional similarity. By updating the query and key matrices through training with large amounts of data, the model encodes in them a notion of relevance that can effectively determine which contexts are important for each token.

The attention scores are then normalized using the softmax function and used to create a weighted sum of the value vectors, putting this together, we get the mathematical formulation for a single self-attention head:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V,$$

where the $\sqrt{d_k}$ scaling factor is present to provide numerical stability, see a graphical representation in Figure 2.8 [20].

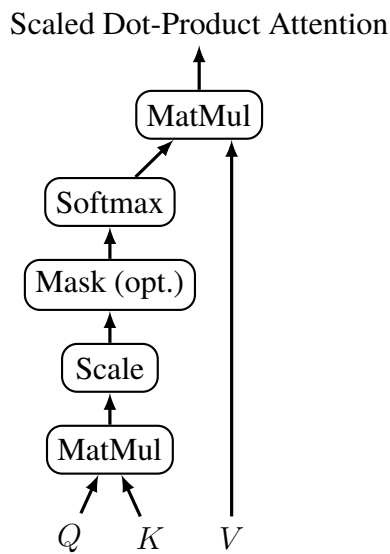


Figure 2.8: Attention head, modified from [21]

This process allows each token's representation to be updated with information from other relevant tokens in the sequence, creating context-aware embeddings that capture the appropriate meaning based on surrounding words.

2.2.2.2 Multi-headed Attention

Multi-headed attention extends the basic attention mechanism by running multiple attention heads in parallel, each with its own learned parameter sets. This approach allows the model to simultaneously attend to information from different representation subspaces, and thus to learn several distinct ways in which context can update the embedding of a token.

Mathematically, multi-headed attention is defined as:

$$\text{MultiHead}(X, Y) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O, \quad (2.2)$$

where each head is computed as:

$$\text{head}_i = \text{Attention}(\underbrace{YW_i^Q}_{Q_i}, \underbrace{XW_i^K}_{K_i}, \underbrace{XW_i^V}_{V_i}),$$

Each head has its own learned projection matrices (W_i^Q, W_i^K, W_i^V) that transform the input embeddings into queries, keys, and values specific to that head. The inputs X and Y are embedding matrices whose specific roles depend on the transformer component being used - we'll clarify these distinctions when we examine the full architecture in Section 2.2.2.4.

Figure 2.9 illustrates the multi-head attention structure.

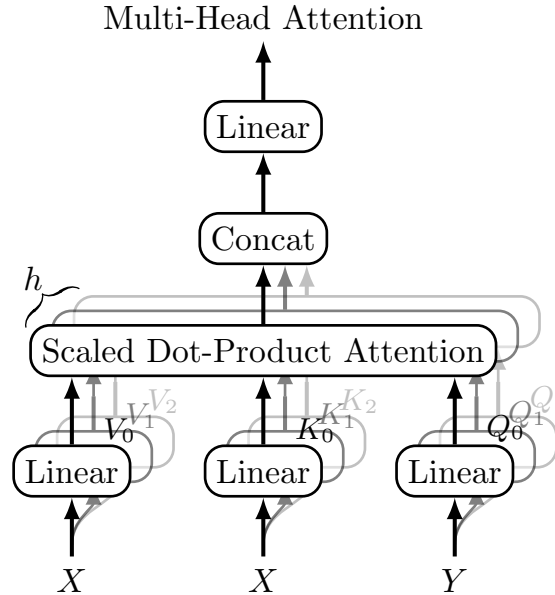


Figure 2.9: Multi-Head attention block, modified from [21]. Where V_i, K_i and Q_i are the value, key and query projections belonging to head_i and h is the number of attention heads

The attention head concatenation followed by linear projection with W^O serves two critical functions:

1. It reshapes the concatenated representations back to the model's required dimension.

2. It learns optimal combinations of the distinct contextual information extracted by each attention head.

This learned transformation enables the model to integrate information from multiple representational subspaces into a unified, context-rich embedding.

Each attention head can learn to focus on different aspects of the input sequence – some heads might capture syntactic relationships, while others focus on semantic connections or positional encoding patterns. This parallel processing of multiple relationship patterns enables a much richer contextual representation than a single attention head could provide.

2.2.2.3 Positional Encoding

Positioning is crucial in language and any other domain where order matters, so capturing positional patterns is indeed desirable for attention mechanisms, as briefly mentioned in Section 2.2.2.2.

However, the transformer architecture as described thus far lacks an inherent mechanism to associate tokens with their positions in a sequence. The attention mechanism alone is permutation-invariant, meaning it treats input tokens as an unordered set.

To address this limitation, Vaswani et al. introduced an elegant positional encoding scheme using trigonometric functions. These encodings generate unique vectors for each position, which are then added to the input embeddings before they enter the transformer. This effectively "embeds" positional information into the token representations [20].

2.2.2.4 Architecture

The transformer architecture stacks alternating attention and feed-forward (linear) blocks in multiple layers, organized into two main components: an encoder and a decoder, see Figure 2.10. The encoder processes the input sequence to create rich contextual representations, while the decoder generates output sequences autoregressively by attending to both its own previous outputs and the encoder's representations.

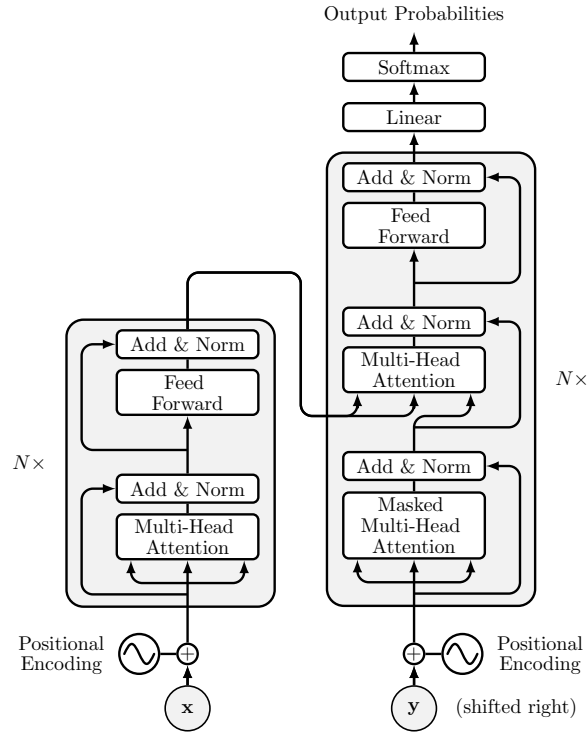


Figure 2.10: Transformer architecture, modified from [21]. The left block is the encoder, and the right block is the decoder.

The transformer architecture employs two distinct types of multi-head attention blocks: self-attention and cross-attention. Self-attention blocks appear in both the encoder and decoder, while cross-attention blocks are used only in the decoder to integrate information between encoder and decoder representations.

Returning to the mathematical description of multi-head attention from Equation 2.2, we can now clarify the meaning of the input matrices X and Y for each attention type:

Self-attention: $Y = X$, where both represent the output embeddings from the previous transformer layer (or word embeddings for the first encoder layer).

Cross-attention: X represents the output of the final encoder layer, while Y represents the current decoder layer output.

In this work, we only employ the encoder block, as we use wav2vec 2.0

(detailed in the next section) which utilizes the transformer encoder in its context network.

2.2.3 wav2vec 2.0

In the context of speech recognition, and many other fields of deep learning for that matter, the scarcity of labeled data is a major bottleneck. Audio data is plentiful, but labeled audio data is less so. Baevski et al. sought to mitigate this problem when they introduced wav2vec 2.0, a novel model architecture that seeks to learn audio structures without labels, and then later fine-tune with labeled data, massively decreasing the requirement for labeled data to achieve good results [22].

wav2vec 2.0 represents a significant advancement in self-supervised learning for speech processing. The model employs a two-stage training paradigm: first, it learns rich representations from raw, unlabeled audio through a contrastive learning objective, then it fine-tunes these representations on downstream tasks with minimal labeled data.

This approach has proven particularly effective for low-resource scenarios, where traditional supervised methods struggle due to insufficient training data.

The architecture consists of three main components that work together to enable effective self-supervised learning, see Figure 2.11. The encoder transforms raw waveforms into latent representations, the quantization module discretizes these representations into learnable speech units, and the context network enriches the latent speech representations with contextual information. During training, the model learns to predict masked portions of the speech signal, developing an understanding of speech structure and phonetic patterns without explicit supervision.

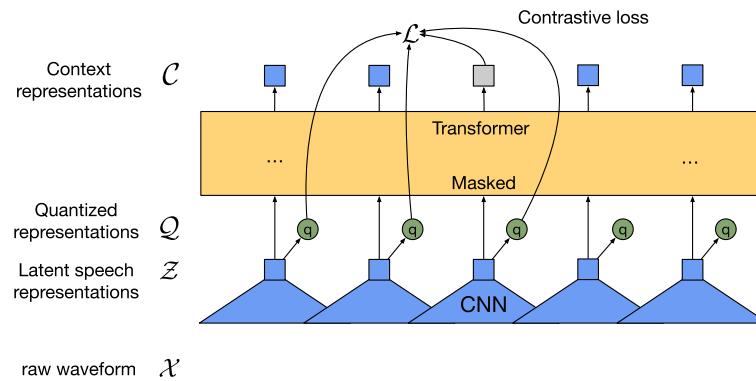


Figure 2.11: wav2vec 2.0 architecture, from [22]

2.2.3.1 Encoder

The encoder consists of convolutional layers that convert raw speech waveforms into latent representations. This convolutional layer acts as a lossy compression algorithm that we train to minimize loss, enabling it to represent the most meaningful audio features in the allotted space. The convolution stride determines the compression factor, effectively downsampling the temporal resolution while preserving essential acoustic information.

2.2.3.2 Quantization Module

The quantization module intuitively serves a role very similar to that of a clustering algorithm. We do not know the different speech units a priori, but we know that there exist different speech units, and therefore different categories of audio data that should match them. The module learns to discretize the continuous latent representations into a finite set of learnable vectors that correspond to distinct speech units.

It does so by learning G codebooks, each with V entries. The model samples and concatenates entries from each group to construct a quantized vector, meaning we can represent V^G discrete units.

To select the codebook entries in a learnable manner, we need a differentiable version of the argmax function, which is achievable with Gumbel-Softmax. This technique enables end-to-end training while maintaining the discrete nature of the quantized representations, crucial for the contrastive learning objective.

Through this process, the quantization module fulfills its primary objective of generating discrete targets for self-supervised pretraining. As illustrated in Figure 2.11, the quantized vectors serve exclusively as targets for loss calculation rather than being propagated forward through the network, effectively simulating the classification targets that enable contrastive learning without requiring labeled data.

2.2.3.3 Context Network

The context network is a modified transformer network that uses relative positional encodings instead of the default absolute positional encodings. This network takes the latent speech representation as input, creating a context-enriched representation for each time step in the latent representation.

The use of relative positional encodings is particularly important for speech processing, as it allows the model to better capture the temporal relationships between different parts of the speech signal regardless of their absolute position in the sequence.

2.2.3.4 Training

The wav2vec 2.0 training objective combines two complementary loss functions:

$$\mathcal{L}_{W2V} = \mathcal{L}_c + \alpha \mathcal{L}_d \quad (2.3)$$

where $\mathcal{L}_c$ is the contrastive loss, and $\mathcal{L}_d$ the diversity loss, with α being a hyperparameter that balances the two terms.

The training process involves masking portions of the latent representations and training the model to identify the correct quantized representation from a set of distractors. This self-supervised approach allows the model to learn meaningful speech representations without requiring transcribed audio data.

Contrastive loss measures how well the model can discern the true quantized latent speech representation from a set of incorrect candidates called distractors when given the context network output. The model learns to maximize the similarity between the context representation and the true quantized target while minimizing similarity to the distractors.

Diversity loss is included to encourage the model to use codebook entries more uniformly, preventing the model from collapsing to a small subset of the available quantized representations.

This training paradigm enables wav2vec 2.0 to learn rich, generalizable representations of speech that can be effectively transferred to downstream tasks with minimal fine-tuning, making it particularly valuable for scenarios where labeled data is scarce.

2.3 Related work

To establish the novelty of applying ASR techniques to SCBD, we conducted a comprehensive literature review of the SCBD field, and its closely related domains. To the best of our knowledge, no prior work has explored ASR-based approaches for SCA, attacks or SCBD.

2.3.1 Side-Channel Based Disassembly

Since the inception of SCBD by Eisenbarth et al. [23], limited attention has been paid to deep learning approaches, with researchers favoring classical statistical methods such as discriminant analysis and support vector machines [9, 10, 24, 25].

To our knowledge, only three research groups have investigated SCBD using deep learning models. Van Geest and Buhan explored multilayer perceptron (MLP) and convolutional neural network (CNN)-based disassembly on STM32F051R8 devices, achieving a modest 25.5% accuracy [26]. Fendri et al. demonstrated more promising results with MLP networks on RISC-V devices, achieving approximately 94% accuracy at $100\times$ oversampling [27].

Alabdulwahab et al. investigated various deep learning architectural blocks for SCBD, including transformer models, though their experiments were conducted entirely on simulated power traces without specified sampling rates, limiting comparability with practical implementations [28].

Notably, none of these works employed speech-to-text models.

2.3.2 Neural Networks in Side-Channel Analysis

Expanding our search to the broader field of SCA reveals successful neural network applications, yet none have leveraged ASR methodologies.

In 2012, Yang et al. demonstrated that standard MLPs are well suited to perform side-channel attacks [29]. Li et al. achieved approximately 90% parameter reduction in CNN-based models while maintaining cryptographic key recovery capabilities, highlighting the potential for efficient neural architectures in side-channel contexts [30]. MLP networks have shown similar success in EM-based analysis, with Khan et al. achieving 100% malware detection rates through EM leakage analysis [4].

2.3.3 Acoustic Side-Channels

Even when examining domains where the physical observables encourage the use of audio based models, ASR techniques remain unexplored.

In the early 2000s, Asonov and Agrawal demonstrated keyboard vulnerability to acoustic side-channel attacks using neural networks for character classification [31]. Zhuang et al. later achieved up to 96% accuracy in character recovery without labeled training data by hidden Markov models for acoustic keyboard analysis [32].

2.3.4 Literature Gap Analysis

Table 2.1 summarizes the current state of SCBD literature, showing variations in accuracy based on a selection of relevant parameters, such as oversampling factor and segmentation approach. Our comprehensive review confirms that ASR techniques have not been applied to SCA in any of the examined domains.

Table 2.1: Comparison of related work on SCBD.

(Src: Source; SC: side-channel type; OSF: oversampling factor; Seg. Type: segmentation type; Sig. Avg: signal averaging; Acc: accuracy)

Src	DUT	SC	OSF	# Instr.	Seg. Type	Sig. Avg	Acc
[8]	AT89S51	EM	1000	90	Trigger	Yes	~97%
[26]	STM32F051R8	Power	156.25	17	Brute Force	Yes	25.5%
[24]	PIC16F15376	EM	500	~50	Windowed	Yes	95%
[27]	GD32VF103	EM	100	34	Trigger	Yes	~94%
[23]	PIC16F687	Power	1000	33	Windowed	No	70%
[9]	MSP430G2553 (TI LaunchPad)	Power	50	152	Template	Yes	~97%
[9]	openMSP430 (Xilinx Spartan 3E)	Power	50	152	Template	Yes	~99%
[10]	ATMega 328P	Power	156.25	112	None	No	99.53%
This Work	Ibex Demo System (CW305-A100)	Power	16	5	Automatic	No	76.61%

Chapter 3

Data Collection Pipeline

The data generation process required developing a comprehensive automated pipeline to systematically collect power consumption data for RISC-V instruction classification. This pipeline needed to integrate instruction sequence generation, compilation, and power trace capture while maintaining experimental consistency and reproducibility across multiple RISC-V instruction types.

All code referenced in this chapter is available in the public GitHub repository [duedahl/riscv-power-tools](#), with specific file references hyperlinked to their respective locations.

3.1 Target System Architecture

The core of our experimental platform is a RISC-V softcore processor deployed on a field-programmable gate array (FPGA). This approach was chosen for its transparency and flexibility. The open nature of the RISC-V ISA allows for a clear understanding of execution mechanisms, while the FPGA's reconfigurability is ideal for hardware-level experimentation. While initial plans included testing multiple hardware-optimized RISC-V implementations, time constraints narrowed the focus to a single, well-defined configuration.

3.1.1 Hardware Setup

The physical setup comprises three main components: a NewAE CW305 board containing a Xilinx Artix-7 FPGA which serves as the target platform, a Digilent JTAG-HS2 programming cable for bitstream flashing and debugging,

and a ChipWhisperer-Husky for acquiring power consumption traces. The complete laboratory configuration is illustrated in Figure 3.1.

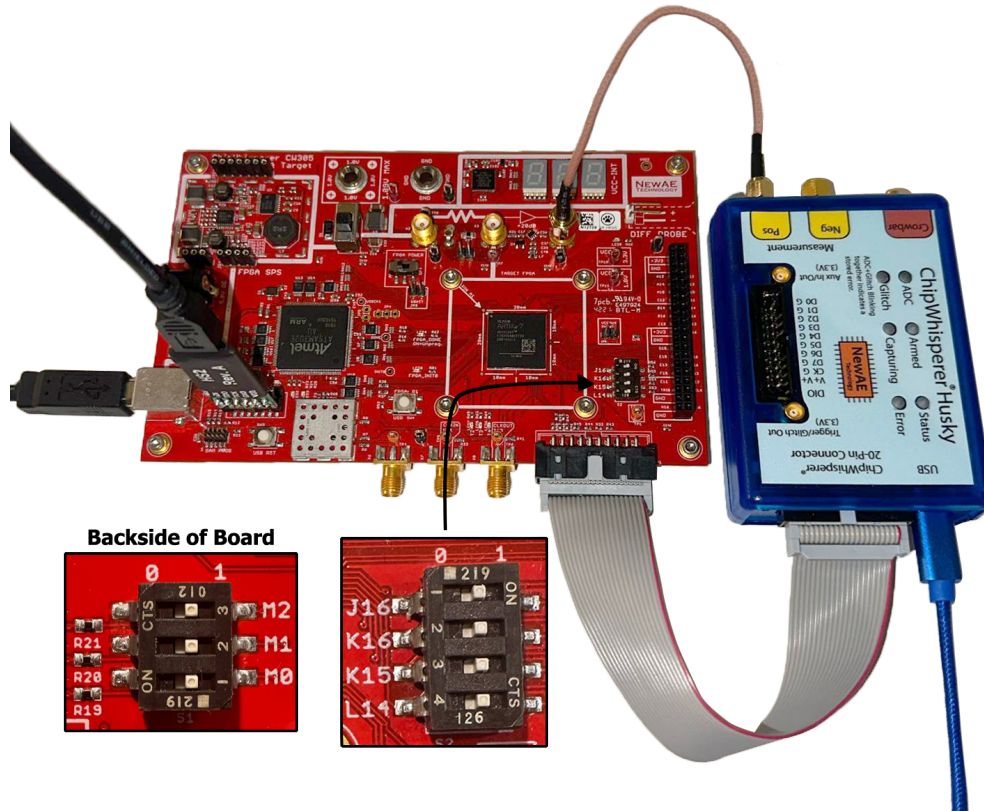


Figure 3.1: Experimental setup consisting of the NAE-CW305-04-7A100-0.10-X FPGA board (left), ChipWhisperer-Husky for power trace capture (right), and Digilent JTAG-HS2 programming cable. Cables extending beyond the frame connect to a host computer.

3.1.2 Softcore Implementation

To execute RISC-V binaries on the FPGA, we deployed the lowRISC Ibex Demo System [33], a system on a chip (SoC) that integrates the Ibex core with essential peripherals like RAM and a JTAG interface as depicted in Figure 3.2.

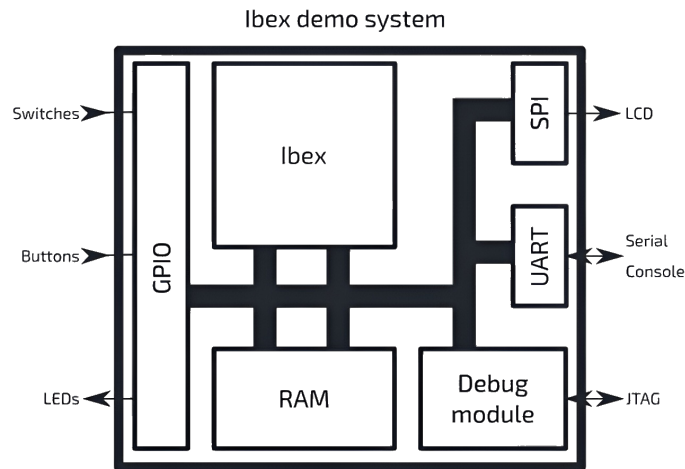


Figure 3.2: Ibex demo system block diagram [33].

The Ibex RISC-V Core serves as the computational engine of the SoC. This open-source 32-bit RISC-V CPU core, developed by lowRISC [34], was configured using the "small" preset, implementing a 2-stage pipeline architecture with instruction fetch in the first stage and combined decode, execute, and writeback operations in the second stage. Figure 3.3 presents the core's pipeline organization.

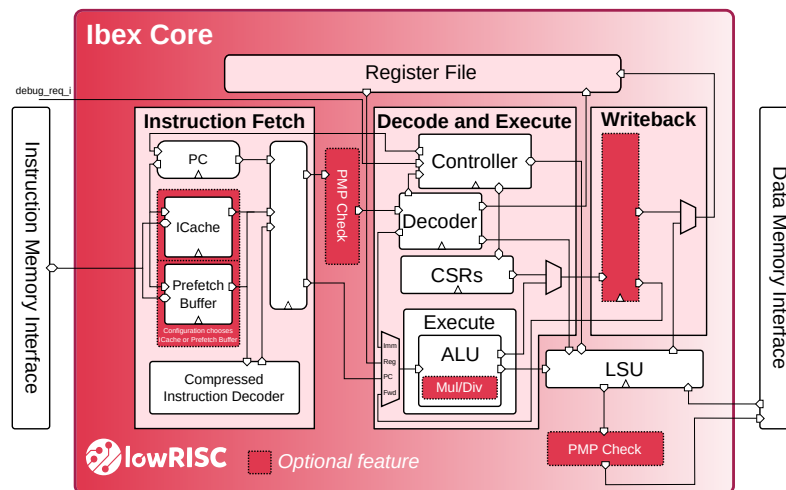


Figure 3.3: Ibex Core pipeline diagram [35].

A critical modification to the default configuration was reducing the Ibex core's clock frequency to 12.5 MHz. This was necessary to ensure

high-resolution power trace captures. Combined with the ChipWhisperer-Husky’s 200 MS/s sampling rate, this frequency yields a 16x oversampling factor, providing sufficient detail to analyze power variations within individual instruction cycles. This level of oversampling strikes a balance between measurement fidelity and the practical challenges of analyzing modern, high-frequency processors.

The frequency was configured by modifying the Phase-Locked Loop (PLL) settings in the SystemVerilog source, following Xilinx guidelines [36, p. 72]:

$$F_{VCO} = F_{CLKIN} \times \frac{M}{D}, \quad F_{OUT} = \frac{F_{VCO}}{O},$$

where M , D , and O correspond to the `CLKFBOUT_MULT_F`, `DIVCLK_DIVIDE`, and `CLKOUT_DIVIDE` parameters, respectively. These were adjusted in the `clkgen_xil7series.sv` file.

A configured bitstream incorporating these modifications is available in the public repository under the filename `ibex_12MHz_512KRAM.bit`

3.1.3 System Interface

To manage the experimental workflow, i.e. flashing the bitstream, programming the softcore via JTAG, and capturing power traces, we developed a custom Python library. This library extends the standard ChipWhisperer Python API [37] with project-specific automation.

The library is implemented in `utils.py`, located in the `Data_Generation_Pipeline` directory. This directory contains all the primary code for this project.

3.2 Data Generation Pipeline

Our approach requires the automated generation of comprehensive power trace datasets for RISC-V instruction analysis. This data generation pipeline encompasses three fundamental stages: generating valid RISC-V 32-bit assembly code, compiling this code into binaries that interface with our trace capture infrastructure, and capturing power traces alongside their associated instruction sequences.

3.2.1 Instruction Sequence Generation

To generate valid RISC-V 32-bit assembly code, we developed a specialized instruction sequence generator (ISG) - a Python library capable of producing randomized instruction sequences with configurable filtering parameters including RISC-V extensions, assembly instruction types, and specific mnemonics.

The development of such an ISG presents several non-trivial challenges beyond ensuring syntactic validity of the instruction sequences. Critical considerations include ensuring return address preservation to prevent program termination, implementing appropriate setup and teardown sequences for specific instructions (which must themselves be randomized to prevent systematic bias), and controlling compiler behavior to prevent unwanted optimizations such as instruction compression when specific mnemonics are required.

Our implementation addresses these challenges through the C file structure presented in Listing 3.1. This structure demonstrates external enforcement of register preservation and the provision of a consistent memory allocation block for load/store operations.

```

1 // Global buffer allocated at compile time with explicit
  alignment
2 static volatile int buffer[1024]
  __attribute__((aligned(8)));
3
4 int execute_cw (void)
5 {
6     // Point to middle of buffer
7     volatile int *buf_ptr = &buffer[512];
8
9     register volatile int *buf_ptr_reg asm("{buffer
      register}") = buf_ptr;
10    register long saved_ra asm("x1");
11
12    // Save return address
13    asm volatile ("mv %0, ra" : "=r" (saved_ra));
14
15    __asm__ volatile (
16        {RISC-V 32-bit asm}
17        :
18        : "r" (buf_ptr_reg)
19        : "memory"

```

```

20     );
21
22     // Restore return address
23     asm volatile ("mv ra, %0" : : "r" (saved_ra));
24
25     return 0;
26 }

```

Listing 3.1: C file structure for each generated software. Where buffer register is a register that will be avoided by the ISG, and the ISG sequence is substituted in place of the braces in the volatile inline assembly block.

The complete implementation of the ISG, including these safety mechanisms and additional functionality, is provided in the files `riscv32_isg.py` and `utils.py`.

3.2.2 Compilation Toolchain

The compilation process extends beyond simple assembly of the generated RISC-V sequences, requiring integration with the ChipWhisperer SDK for subsequent trace capture operations. We achieve this integration through a common `main.c` file that utilizes the NewAE-provided `simpleserial.h` header, enabling access to their `simpleserial` protocol SDK.

Listing 3.2 illustrates the integration between the `simpleserial` SDK and our generated code. The `execute_cw()` function from Listing 3.1 is bracketed by `trigger_high()` and `trigger_low()` calls, which provide precise timing control for the Husky trace capture system.

Note however, this is a simplified main file, the original can be found in the public GitHub repository, and contains a number of compiler directives and debugging instrumentation.

```

1 #include "simpleserial.h"
2 uint8_t perform_function(uint8_t* selection, uint8_t len)
3 {
4     trigger_high();
5     execute_cw();
6     trigger_low();
7     return 0;
8 }

```

Listing 3.2: Simplified section of the `main.c` file showing interplay between the `simpleserial` SDK and our `execute_cw()` function

Our compilation infrastructure builds upon the `ibex-demo-system` C-stack build system, with modified Makefiles to accommodate our specific software requirements. This compilation stack supports the RISC-V 32-bit base, multiplication, and compressed (RV32IMC) extensions. Implementation details for this compilation process are documented in the `utils.py` library.

3.2.3 Data Generation

With compiled binaries available, we proceed to program them onto the softcore processor, label the instruction sequences, and capture power traces during execution. The programming, labeling, and capture methods are implemented in `utils.py` and support two distinct labeling approaches: hardware debugging and simulation.

3.2.3.1 Hardware Debugging

Our debugging methodology leverages the `ibex-demo-system` debugging infrastructure through OpenOCD to establish a debugging server.

Using Python's multiprocessing in conjunction with `pexpect`, we automatically step through program execution from `trigger_high()` to `trigger_low()`, logging each intermediate assembly instruction to establish ground truth labels.

This approach, while providing the highest fidelity results, exhibits limited throughput, typically processing approximately 10 assembly instructions per second on our experimental setup. This performance constraint necessitates alternative approaches for large-scale data generation.

3.2.3.2 Simulation

To address the throughput limitations of hardware debugging, we employ the Spike RISC-V ISA Simulator, supported by lowRISC [38], to rapidly simulate instruction sequence execution on the `ibex-demo-system`. This simulation approach requires extraction of memory addresses for the `trigger_high()` and `trigger_low()` symbols, enabling instruction logging between these boundaries using methodology analogous to the debugging approach.

The simulation approach provides significant performance improvements while maintaining instruction-level accuracy, making it suitable for large-scale dataset generation and rapid prototyping of instruction sequences.

3.3 Data Validation

The reliability of our later model training depends critically on the accuracy of both instruction labels and captured power traces. Therefore, we assess the validity of both labels and power traces before proceeding with analysis.

3.3.1 Label Validation

Given our reliance on Spike simulation for instruction labeling, we must verify the correctness of these labels against ground truth data. We conducted this validation by generating an initial dataset covering all RV32IMC instructions, then comparing Spike-generated labels against those obtained through hardware-level debugging.

This validation revealed that Spike simulations provide accurate instruction-level labeling with one notable exception: when an instruction corresponds to a pseudoinstruction, the simulator may yield different mnemonics compared to hardware debugging. However, this discrepancy does not affect the functional accuracy of the simulation, and Spike operates correctly for all RV32IMC instructions implemented in our ISG.

3.3.2 Trace Validation

Our trace capture methodology produces power traces with a predictable three-part structure: a prefix containing setup instructions, the target inline assembly instruction, and a suffix containing teardown instructions. The prefix and suffix segments derive from the instrumentation framework surrounding our inline assembly and remain nearly identical across all traces.

Some variation in these segments occurs due to compiler-dependent register saving and restoration requirements, but this variation is minimal and systematic. Consequently, we expect to observe highly consistent head and tail regions across all captured traces.

Figure 3.4 demonstrates this consistency by displaying superimposed power traces for all RV32I single-instruction captures. The traces are aligned at both head and tail positions to facilitate visual inspection of the prefix and suffix code consistency.

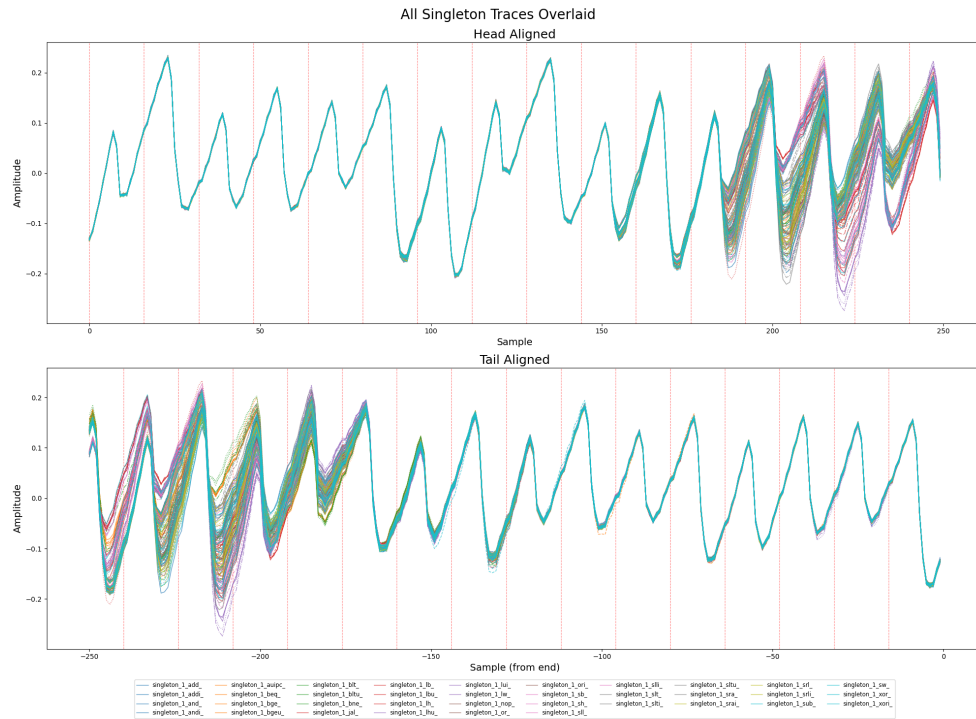


Figure 3.4: Overlaid power traces for several repetitions of RV32I single instructions including the prefix and suffix code associated with the ChipWhisperer instrumentation, shown with both head- and tail-alignment to demonstrate consistency.

The extreme consistency observed for identical instructions enables effective trace validation through visual inspection. For each generated dataset, we perform quality control by examining the trace headers for any aberrations that might indicate capture errors or instrumentation failures.

While visual inspection proves sufficient for our current dataset sizes, we acknowledge that automated trace validation prior to prefix and suffix removal would enhance scalability and reproducibility. Implementation of such automated validation represents a natural extension of this work for larger-scale data generation efforts.

3.4 Data Preprocessing

The prefix and suffix regions of our captured power traces, while necessary for proper instrumentation, introduce systematic bias into the dataset and must be removed prior to analysis. These regions contain setup and teardown code that is largely identical across all traces and would otherwise dominate any machine learning model’s decision-making process.

Our preprocessing pipeline removes these regions through a systematic length-based approach. We first determine the trace lengths corresponding to individual instructions within both the prefix and suffix regions. Additionally, we capture traces with no inline assembly instructions provided, allowing us to account for any trace components that cannot be directly mapped to specific instructions.

This methodology enables precise determination of prefix and suffix boundaries, which we then use to prune the traces accordingly. The resulting preprocessed traces contain only the power consumption signatures of the target instructions, eliminating the instrumentation artifacts that would otherwise bias subsequent analysis.

Implementation details for this preprocessing pipeline are provided in the [generateData.ipynb](#) notebook.

3.5 Data Inspection

To explore the characteristics of our power trace dataset and assess the distinguishability of different instruction types, we conducted a visual inspection of the captured traces. This exploratory examination reveals distinct patterns in power consumption that seem associated with instruction format groupings.

3.5.1 Intra-Group Similarity

Power traces exhibit remarkable similarity when comparing instructions within the same format group. Figure 3.5 demonstrates this phenomenon for RV32I I-Type instructions, where all traces display nearly identical power consumption. This consistency extends across all instruction format groups, as shown in the visualizations provided in Appendix A.

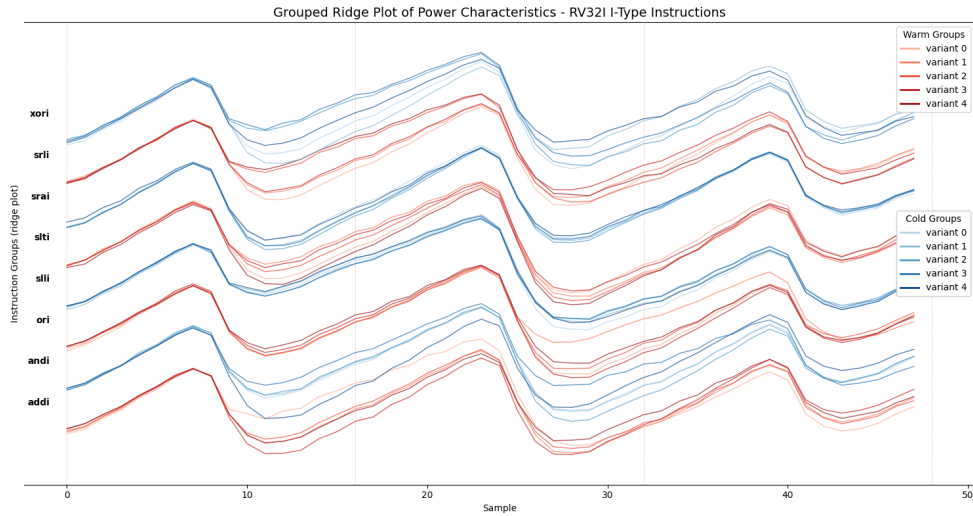


Figure 3.5: Ridge plot of power traces for RV32I I-Type instructions. For each instruction, five variants were generated with randomly selected operand registers and immediates. To aid in visual interpretation, the traces for each instruction are partitioned into two color-distinguished groups in an alternating sequence.

3.5.2 Inter-Group Distinguishability

In contrast to the intra-group similarity, comparison across different instruction format groups reveals tangible differences in power consumption patterns. Figure 3.6 presents a selected subset of five mutually distinguishable RV32I instructions, chosen specifically for their distinct power signatures. A comprehensive comparison encompassing all instruction types is provided in Figure A.5.

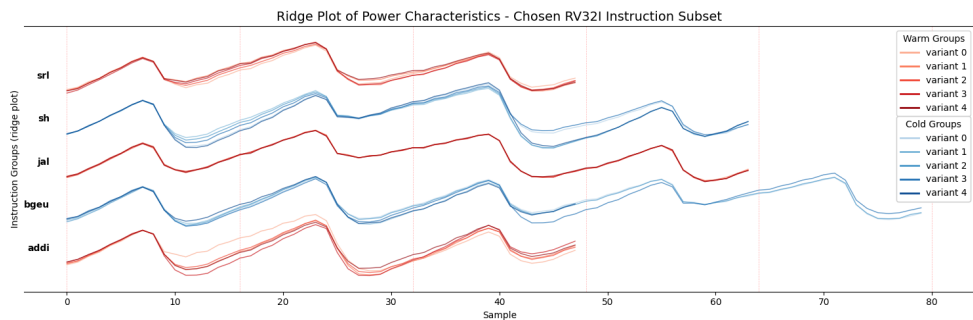


Figure 3.6: Ridge plot of the chosen subset of five mutually distinguishable RV32I instructions.

3.5.3 Instruction Set Downscoping

The 16× oversampling available in our experimental setup provides limited resolution for distinguishing between instructions within the same instruction format group. A pilot experiment attempting SCBD using the full RV32IMC instruction set confirmed that reliable discrimination between same-format instructions is indeed challenging with our current measurement resolution.

Following this result, our visual inspection of the data revealed the underlying issue: instructions within the same format group exhibit nearly identical power consumption patterns, while instructions from different format groups show tangible differences.

Consequently, we focused our analysis on the subset of five clearly distinguishable instructions, [srl, sh, jal, bgeu, addi], shown in Figure 3.6.

This reduced scope serves as a proof-of-concept evaluation, testing whether our chosen automatic speech recognition model can perform both segmentation and classification tasks without requiring feature engineering or manual intervention.

However, this scope reduction correspondingly limits the potential impact of our findings, raising the question of whether higher oversampling rates would enable broader instruction set coverage and thus more meaningful comparisons with existing literature.

Chapter 4

Model Development

This chapter describes the adaptation, training, and evaluation methodology for applying ASR techniques to side-channel power trace analysis. Our approach adapts the Wav2Vec 2.0 architecture to the domain of SCBD by way of hyperparameter optimization across multiple training phases.

All code developed for training and evaluating our model is available in the public Github repository [duedahl/asr4scbd](https://github.com/duedahl/asr4scbd).

4.1 Model Adaptation

We adapt the wav2vec 2.0 pretraining implementation from the HuggingFace transformers library [39] to accommodate side-channel power trace data. Our implementation builds upon the existing [HuggingFace speech pretraining framework](#), with modifications to handle the unique characteristics of power trace inputs.

The adaptation process involves minimal architectural changes to the base wav2vec 2.0 model, focusing instead on domain-specific adjustments for SCBD tasks. Key modifications include input preprocessing pipelines for power trace data and output layer adaptations for assembly instruction prediction.

Additionally, we developed a separate finetuning framework that trains the final fully connected layer without the quantization module, using labeled training data and CTC loss to optimize the model for specific instruction recognition tasks.

Both pretraining and finetuning implementations are instrumented with OmegaConf for declarative hyperparameter configuration management, allowing reproducible experiments through YAML configuration files. Fur-

thermore, we integrate Optuna for intelligent hyperparameter search [40], enabling systematic exploration of the hyperparameter space across both training phases.

4.1.1 Tokenization Strategy

We develop custom vocabularies tailored to each dataset, where each vocabulary consists of all unique assembly instructions present in the corresponding dataset. Unlike traditional ASR applications where words are often decomposed into sub-word tokens, we treat assembly instructions as atomic tokens. This design choice reflects the fundamental difference between our domain and speech recognition: in SCBD, each assembly instruction in a sequence corresponds directly to a "word" in the ASR framework, but these instructions represent indivisible semantic units that should not be further decomposed.

Our tokenization scheme employs three special tokens alongside the instruction tokens:

[PAD] : Padding token required for length normalization

[UNK] : Unknown token for out-of-vocabulary instructions during inference

▮ : Delimiter token used to separate instructions

The delimiter token is particularly important as it preserves the sequential structure necessary for accurate word error rate (WER) calculations during evaluation.

4.2 Dataset Generation

All datasets in this work use the `rvi_mdb` naming convention, indicating they contain only the chosen RV32I (m)utually (d)istinguishable (b)asis instructions described in Section 3.5.3.

We generate datasets in two size variants - reduced and full - for both pretraining and finetuning phases. The reduced datasets serve as subsets of their corresponding full datasets, enabling efficient hyperparameter exploration before scaling to the complete training regime. Table 4.1 provides detailed specifications for each dataset variant.

Table 4.1: Dataset metadata for rvi_mdb variants. Where # Seq is the number of different sequences generated for each length and # Rep is the number of power traces captured per sequence. All datasets are available on Hugging Face, each dataset name is a hyperlink to their corresponding repository.

Phase	Size	Seq. Lengths	# Seq	# Rep	Name
Pretrain	Reduced	200, 250, ..., 350	100	5	pre_rvi_mdb
	Full	100, 150, ..., 350	1500*	5	full_pre_rvi_mdb
Finetune	Reduced	5, 10, ..., 20	100	5	rvi_mdb
	Full	5, 6, ..., 20	100	5	full_rvi_mdb

The pretraining datasets focus on more and longer sequences (100-350 instructions), while the finetuning datasets use fewer and shorter sequences (5-20 instructions) to reflect the reality that labeled training data is often more troublesome to obtain. Furthermore, for inference, it doesn't make too much of a difference (other than potentially throughput) whether we can process long sequences, as we can just process it in chunks. As such, it seems the pertinent choice to just focus on good accuracy in shorter sequences.

Each dataset is partitioned into random train-validation-test splits with proportions 70-15-15, ensuring that each unique instruction sequence appears in only one split. This enables proper model evaluation by preventing data leakage between training, validation, and test sets.

All datasets are publicly available on Hugging Face in [this collection](#).

4.3 Hyperparameter Optimization Framework

To perform hyperparameter tuning, we use Optuna, a framework that automates the search for optimal hyperparameter values [40].

Optuna defaults to using the Tree-Structured Parzen Estimator (TPE) algorithm, a Bayesian optimization method that uses historical trial results to intelligently guide the search toward promising hyperparameter regions [41].

For the pretraining phase we tune the parameters seen in Table 4.2. Notably, a number of these parameters affect the wav2vec 2.0 architecture.

*This is in addition to those in the pre_rvi_mdb subset.

Table 4.2: Hyperparameter search space for pretraining optimization. Square brackets denote a range, and curly braces denote a set.

Parameter	Type	Range/Options
learning_rate	Float (log)	$[10^{-4}, 10^{-2}]$
mask_time_prob	Float (log)	$[0.05, 0.4]$
mask_time_length	Integer	$[1, 8]$
num_hidden_layers	Categorical	$\{6, 12\}$
num_codevectors_per_group	Categorical	$\{4, 6, 8\}$
num_codevector_groups	Categorical	$\{1, 2\}$
diversity_loss_weight	Float	$[0.5, 2.0]$
conv_stride	Categorical	$\{[4, 2, 2], [2, 2, 1], [2, 1, 1]\}$

The `conv_stride` parameter controls the compression factor in the encoder block of the wav2vec 2.0 architecture (Section 2.2.3.1), while `diversity_loss_weight` corresponds to the α factor from Equation 2.3 that balances contrastive and diversity losses. The codevector parameters configure the codebook structure in the quantization module (Section 2.2.3.2), and the `mask_time` parameters govern the masking strategy during pretraining (Section 2.2.3.4).

For the finetuning phase, we tune the parameters shown in Table 4.3, which consist primarily of standard training hyperparameters. The exception is `freeze_feature_encoder`, which determines whether the wav2vec 2.0 feature encoder should remain frozen or continue training during the finetuning process.

Table 4.3: Hyperparameter search space for finetuning optimization. Square brackets denote a range, and curly braces denote a set.

Parameter	Type	Range/Options
learning_rate	Float (log)	$[10^{-5}, 10^{-3}]$
per_device_train_batch_size	Categorical	$\{32, 64, 128, 256\}$
weight_decay	Float	$[0.001, 0.1]$
warmup_steps	Integer	$[50, 500]$
gradient_accumulation_steps	Categorical	$\{1, 2\}$
freeze_feature_encoder	Categorical	$\{\text{True}, \text{False}\}$

4.4 Training Strategy

Our training methodology follows a systematic hyperparameter optimization strategy designed to identify optimal configurations across both pretraining and finetuning phases. The approach, illustrated in Figure 4.1, employs a multi-stage process that balances computational efficiency with thorough exploration of the hyperparameter space.

The strategy begins with Optuna-based hyperparameter tuning on the reduced pretraining dataset (`pre_rvi_mdb`) to identify promising parameter configurations. From these results, we select the most promising hyperparameter set and create four distinct branches (A, B, C, D) that explore variations around this optimal configuration. This branching strategy allows us to investigate the robustness of our findings while maintaining computational tractability.

Each branch undergoes comprehensive pretraining on the reduced dataset, after which the final linear layer is initialized through brief finetuning on the reduced finetuning dataset (`rvi_mdb`). These initialized models then serve as starting points for a second round of Optuna optimization, now targeting finetuning hyperparameters. This two-stage approach provides a computationally efficient way to optimize configuration of both pretraining and finetuning phases for our specific domain, compared to end-to-end hyperparameter tuning.

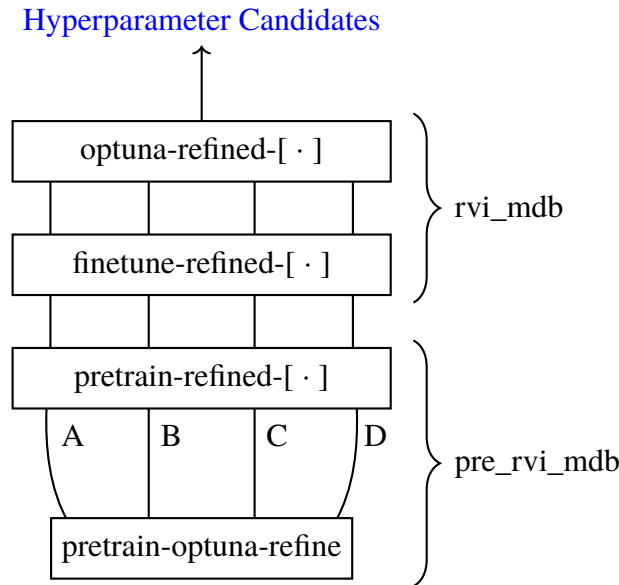


Figure 4.1: Overview of hyperparameter tuning strategy. Each rectangle represents a different stage, where $[\cdot]$ is the placeholder for the associated branch character label. The braces on the right denote the data sets used for the stages they span.

Upon completion of the hyperparameter optimization process, we obtain four distinct sets of optimized pretraining and finetuning hyperparameters corresponding to branches A through D. These optimized configurations are then used to conduct full-scale training runs on the larger datasets (full_pre_rvi_mdb and full_rvi_mdb), providing the final model candidates for evaluation and comparison.

4.5 Evaluation Metrics

We employ two complementary evaluation metrics to assess model performance: WER for measuring transcription accuracy, and a modified McNemar test adapted for transcription tasks to establish statistical significance.

4.5.1 Word Error Rate

The WER serves as our primary evaluation metric, adapted from the automatic speech recognition domain where it represents the standard measure of transcription quality. WER quantifies the difference between a predicted

instruction sequence (hypothesis) and the ground truth sequence (reference) at the word - or in our case, instruction - level.

WER is computed by determining the minimum edit distance between the hypothesis and reference sequences, counting the number of substitutions (S), insertions (I), and deletions (D) required to transform the reference into the hypothesis [42, ch. 16.5]:

$$WER = \frac{I + S + D}{N} = \frac{I + S + D}{C + S + D}, \quad (4.1)$$

where C represents correct predictions and N is the total number of instructions in the reference sequence. A WER of 0.0 indicates perfect transcription, while higher values indicate increasing error rates.

4.5.2 Segmentation Error Rate

We introduce the segmentation error rate (SER) to specifically evaluate the model's performance on the segmentation task, independent of its classification accuracy. An error in segmentation occurs when the model fails to correctly identify the segment boundaries, leading to an insertion or a deletion. We therefore define SER by adapting Equation 4.1 to exclude substitution errors from the error count, as a substitution represents a classification error, not a segmentation one:

$$SER = \frac{I + D}{N} = \frac{I + D}{C + S + D}.$$

This metric allows us to quantify how well the model identifies the presence and structure of instruction segments, which is a distinct task from classifying the content within them.

4.5.3 ASR Adapted McNemar

To establish statistical significance of our model's performance improvements, we adapt McNemar's test to handle the specific challenges of sequence transcription tasks. Traditional McNemar's test compares two classifiers by examining their disagreements, but standard formulations do not account for the insertion and deletion errors inherent in transcription models.

Our ASR Adapted McNemar test addresses this limitation by incorporating all edit operations (substitutions, insertions, deletions) into the significance testing framework. The test compares our model against a naive majority

class baseline that can only make substitution errors, providing a conservative statistical comparison.

The key decision in our approach lies in the assignment of insertion errors to the contingency table. Rather than assigning insertions in a neutral manner, we deliberately bias the test against our own model by placing insertion errors in the position that minimizes the test statistic. This conservative approach ensures that any claimed statistical significance is robust and not artificially inflated.

The test maintains the standard McNemar assumption that errors should be independent. While this assumption is often violated in speech recognition due to contextual dependencies [42, ch. 16.5], our domain of independent instruction sequences with corresponding power traces makes this assumption more reasonable, though not guaranteed.

Detailed mathematical derivations, including the construction of the modified discordant contingency table entries and the bias assignment procedure, are provided in Appendix B.

Chapter 5

Results and Analysis

In this chapter, we present the results from our multi-stage hyperparameter tuning approach and analyze the performance of our power trace disassembly models.

5.1 Hyperparameter Tuning

In the first stage of our tuning strategy, we systematically explored the pretraining parameter space. Figure 5.1 presents the parameter performance across different Optuna trials in a slice plot format.

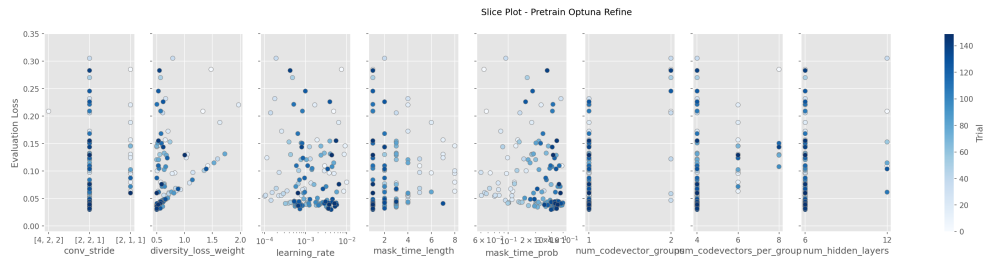


Figure 5.1: Slice plot of hyperparameters for pretrain Optuna trials, cropped to remove outliers for clarity.

Several parameters exhibit clear optimization trends. The `conv_stride` parameter shows a strong preference for the `[2,2,1]` configuration, which is unsurprising given the options, as we unfortunately left out the `[4,2,1]` configuration from the search space.

The `[4, 2, 2]`, `[2, 2, 1]` and `[2, 1, 1]` stride configurations correspond to compression factors of 16, 4 and 2 of the input. And with an oversampling rate

of 16x within an instruction cycle, it is consistent with the poor performance of [4, 2, 2] as it would compress to the granularity of full instruction cycles, not leaving room for the model to learn from patterns within the cycle. This compression level essentially discards the fine-grained power variations within instruction cycles that enable the model to distinguish between similar instructions.

The minimal compression of [2, 1, 1] may be counterproductive to training efficiency by preserving excessive detail, requiring the model to process more timesteps without proportional benefit in discriminative power.

The `diversity_loss_weight` consistently trends toward the lower boundary of our search space, suggesting that we excessively constrained its search space. This pattern indicates that a rerun with a relaxed or eliminated lower boundary could yield better performance. However, this constraint was intentionally implemented based on pilot experiments where insufficient diversity weighting caused training collapse, with the model degenerating into using only a single codeword from the entire codebook.

This creates a delicate optimization challenge: the diversity loss must be strong enough to prevent codebook collapse while remaining minimal enough to avoid interfering with the primary contrastive learning objective. The consistent boundary-seeking behavior suggests our current lower limit may be too conservative, but any adjustment would require careful monitoring to ensure codebook utilization remains healthy.

Another interesting finding is the model's preference for small codebook configurations, as evidenced by the trends in `num_codevector_groups` and `num_codevectors_per_group`. However, this trend may be partially artefactual rather than indicative of optimal model capacity. Smaller codebooks inherently have lower upper bounds for diversity loss - reducing codebook size inherently reduces the diversity loss component of the objective function, potentially biasing the optimization toward smaller configurations.

This highlights a limitation of our stage-wise tuning approach: codebook size parameters are fundamentally interdependent with the diversity loss weighting and cannot be optimally determined in isolation. The true test of codebook configuration should be evaluated in an end-to-end context where final task performance, measured by WER or CTC loss, determines the optimal trade-off between codebook expressiveness and training stability.

The number of hidden layers shows no clear benefit from increases, which may indicate either that the representational complexity of power traces

doesn't require deep architectures, or that our training duration was insufficient to leverage the additional capacity.

To quantify the relative impact of each hyperparameter, we employed Optuna's fANOVA based visualization tool, with results shown in Figure 5.2.

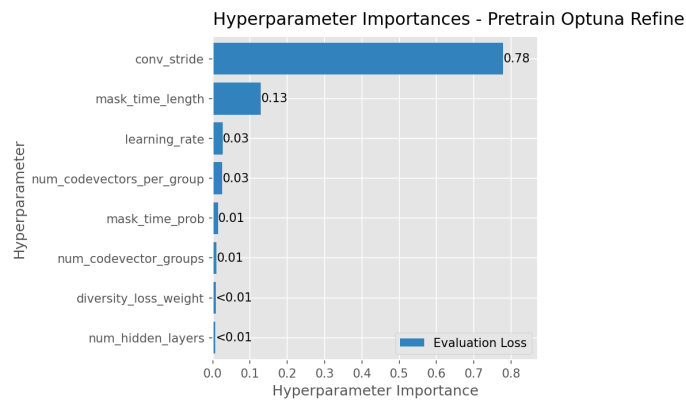


Figure 5.2: Pretraining Optuna parameter importance plot.

Convolutional stride lengths and masking length emerge as the most critical parameters. The stride importance is expected given its direct impact on the compression ratio applied to our input sequences. With an oversampling rate of 16, compression factors exceeding this value cross a critical threshold - rather than merely condensing information, they begin to discard the fine-grained temporal patterns that distinguish different instructions within each cycle.

The masking length's high importance reflects its fundamental role in the self-supervised learning objective. Suppose compression occurs at a factor of 16 and masking affects 3 timesteps, we effectively mask 3 complete instruction cycles, making it impossible for the model to infer the middle instruction from neighboring traces.

An optimal masking strategy should partially mask instruction cycles rather than completely obscuring them. This approach allows the model to leverage both internal instruction patterns and the subtle signal artifacts that bleed between adjacent instruction boundaries. Such partial masking creates a learnable reconstruction task where the model can develop robust representations by inferring missing portions from available contextual cues, rather than facing an impossible prediction challenge.

5.1.1 Configuration Selection

Based on our analysis, we selected seed configurations for further exploration. Figure 5.3 shows the training metrics for our best-performing trials, with the two selected models marked in red and green.

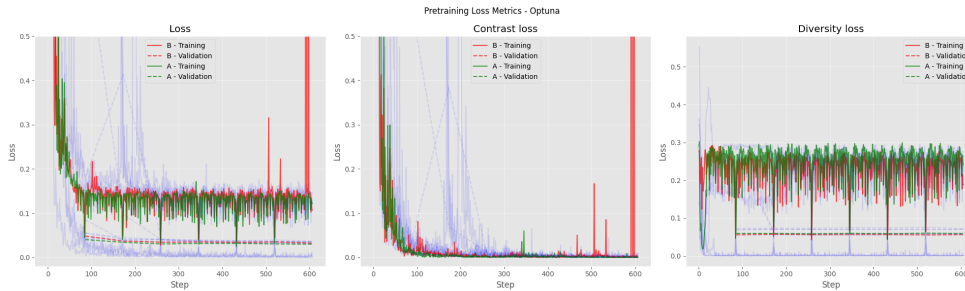


Figure 5.3: Pretraining loss for multiple Optuna trials, with seed runs for branch A and B marked.

This visualization revealed critical insights about training dynamics. Multiple trials showed clear overfitting patterns, with training losses substantially lower than validation losses. Our selected models, however, maintained healthy train-validation gaps, indicating they had not yet reached their learning capacity.

An unexpected finding was the regular loss spikes occurring with epoch-level periodicity. Investigation revealed these originated from our batch sampling strategy, which grouped training examples by sequence length to minimize padding overhead. This approach inadvertently created deterministic batch compositions and ordering, leading to the observed regularity in diversity loss fluctuations.

We resolved this issue for subsequent training runs, though we determined it was non-critical to the validity of our initial hyperparameter exploration.

Table 5.1 documents the final configurations for our selected models. We manually standardized the codebook configuration to 8 codevectors per group across 2 groups, ensuring sufficient vocabulary capacity for our instruction set plus special tokens.

Table 5.1: Base model configurations. Models C and D are variants of Model A with modified convolutional strides: [4, 2, 1] and [4, 2, 2] respectively. All models use 8 codevectors per group, 2 codevector groups at 6 hidden layers.

Parameter	Model A	Model B
Learning Rate	4.18×10^{-3}	1.5×10^{-4}
Conv Stride	[2, 2, 1]	[2, 2, 1]
Mask Time Length	1	2
Mask Time Probability	0.3	0.33
Div. loss weight	0.5	0.56

Given Model A’s superior performance, we used it as the foundation for branches C and D, modifying only the convolutional stride to [4,2,1] and [4,2,2] respectively. This targeted exploration allows us to systematically investigate the search space around our most impactful parameter.

With branch parameters established, we proceeded to extended pretraining runs. Training progressed without noteworthy events, so the metrics are relegated to Figure C.1 in the appendix.

5.1.2 Finetuning Optimization

Following extensive pretraining with our refined configurations, we proceeded to finetuning optimization. The initialization stage revealed an intriguing pattern: Branch B demonstrated significantly faster initial learning compared to all A-derived configurations (Figure 5.4).

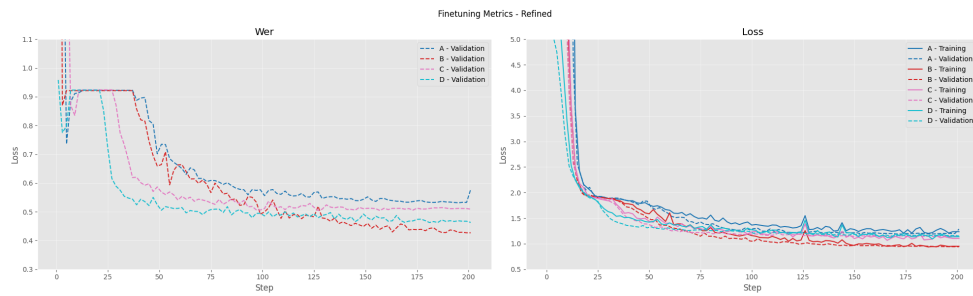


Figure 5.4: Finetuning initialization metrics for each branch.

The final finetuning optimization yielded the hyperparameters shown in Table 5.2.

Table 5.2: Best Optuna hyperparameter configuration for branches A–D

Parameter	Model A	Model B	Model C	Model D
Learning Rate	6.22e-4	3.10e-4	4.44e-4	4.44e-4
Per Device Train Batch Size	64	32	128	32
Weight Decay	0.0296	0.0489	0.0015	0.0710
Warmup Steps	298	332	82	472
Gradient Accumulation Steps	2	1	2	2
Freeze Feature Encoder	False	False	False	False

Branch C distinguishes itself by having `freeze_feature_encoder` rather than learning rate as the most important parameter (see Appendix C). This suggests that for the [4,2,1] convolutional stride configuration, there may be substantially more potential for improving internal representations compared to other branches.

5.2 Training Metrics

5.2.1 Pretraining

Figure 5.5 presents the complete pretraining metrics for all model branches, revealing distinct learning patterns across configurations.

Branch A exhibits the expected learning behavior: steady improvement in contrastive loss without overfitting, and stable diversity loss indicating appropriate codebook utilization without forced diversity.

Branches B, C, and D display markedly different characteristics. All three show substantial gaps between training and validation losses for both contrastive and diversity components.

Notably, the training loss consistently exceeds validation loss across all metrics, which, while not unreasonable given our regularization techniques (dropout, weight decay), contrasts sharply with Branch A’s behavior.

The large initial loss drops in branches C and D suggest suboptimal initialization, though this appears to have minimal impact on validation performance. Branch D’s discrete drops in diversity loss are particularly interesting, suggesting the model progressively incorporates additional codewords into its active vocabulary during training.

These sudden vocabulary inclusions also correlate with increases in contrast loss, suggesting that the diversity to contrast loss ratio is suboptimal.



Figure 5.5: Metrics for the full pretraining of models A–D.

5.2.2 Finetuning

The finetuning results (Figure 5.6) reveal that all branches begin overfitting relatively early, though at different rates.

Despite this, all models achieve similar WER performance, converging to the 30–35% range. Note, our metrics calculations did not account for padding tokens present during training, artificially inflating the reported WER values. The corrected evaluation metrics are presented in the following section.

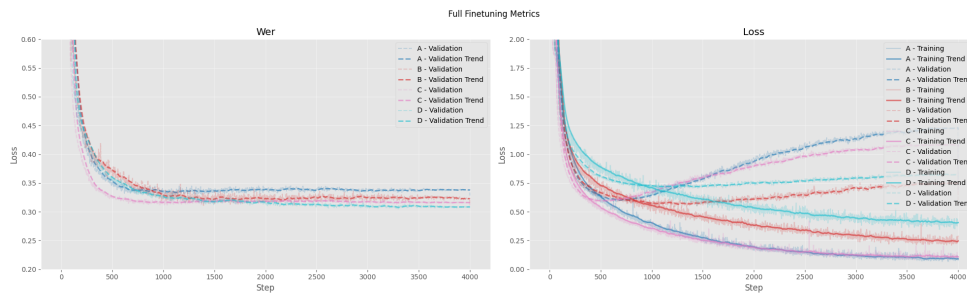


Figure 5.6: Metrics for the full finetuning of models A–D.

5.3 Evaluation

Table 5.3 presents the WER results for all dataset splits, the train and val split are included to show their performance at evaluation time when padding is not an issue.

Table 5.3: WER results for branches A–D across all splits.

Dataset	Model A	Model B	Model C	Model D
Train	0.1360	0.1387	0.1089	0.1274
Val	0.2601	0.2369	0.2459	0.2426
Test	0.2604	0.2392	0.2339	0.2416

The corrected metrics reveal substantially better performance, with Model C achieving the lowest WER of 23.4% on the test set. This represents a significant improvement over the uncorrected values and demonstrates that our models are successfully learning to overcome the segmentation challenge while maintaining high classification accuracy.

5.3.1 Statistical Validation

To establish statistical significance, we performed ASR-adapted McNemar tests comparing each model against a naive baseline capable of segmentation with majority-class classification. Table 5.4 presents the results.

Model A	Naive Baseline	
	Correct	Incorrect
Correct	1981	6268
Incorrect	593	2217
$\chi^2 = 4692, p < 0.001$		
(a) Finetune Full A		

Model B	Naive Baseline	
	Correct	Incorrect
Correct	2099	6378
Incorrect	474	2107
$\chi^2 = 5085, p < 0.001$		
(b) Finetune Full B		

Model C	Naive Baseline	
	Correct	Incorrect
Correct	2153	6423
Incorrect	462	2062
$\chi^2 = 5159, p < 0.001$		
(c) Finetune Full C		

Model D	Naive Baseline	
	Correct	Incorrect
Correct	2212	6255
Incorrect	377	2230
$\chi^2 = 5208, p < 0.001$		
(d) Finetune Full D		

Table 5.4: ASR Adapted McNemar test results for all four model variants of the "Finetune Full" run, against the naive baseline. Each 2×2 contingency table shows the agreement/disagreement between the model and baseline, with corresponding chi-square statistics and p-values.

All models demonstrate statistically significant improvements over the baseline ($p < 0.001$), with χ^2 values exceeding 4000.

5.3.2 Segmentation Error Analysis

To isolate segmentation quality from overall transcription performance, we computed SER values that exclude substitution errors and focus solely on temporal alignment accuracy. Table 5.5 presents the SERs across all model variants and dataset splits.

Table 5.5: SER results for branches A–D across all splits.

Dataset	Model A	Model B	Model C	Model D
Train	0.0430	0.0355	0.0258	0.0821
Val	0.0945	0.0760	0.0788	0.1273
Test	0.0791	0.0686	0.0708	0.1143

The segmentation error rates are substantially lower than their corresponding WER values. Comparing segmentation and overall error rates in Tables 5.5 and 5.3 reveals that segmentation accounts for approximately a third of total errors across models, indicating that the primary challenge lies in instruction classification rather than temporal alignment. This suggests that our models have largely mastered the segmentation task, with remaining errors predominantly arising from confusion between instructions with similar power consumption patterns.

5.3.3 Error Pattern Analysis

To understand the types of errors our models make, we analyzed the confusion patterns between predicted and actual instructions. Figure 5.7 shows the confusion matrix for Model C, our best-performing variant.

Confusion matrices for the remaining models can be seen in Appendix D.

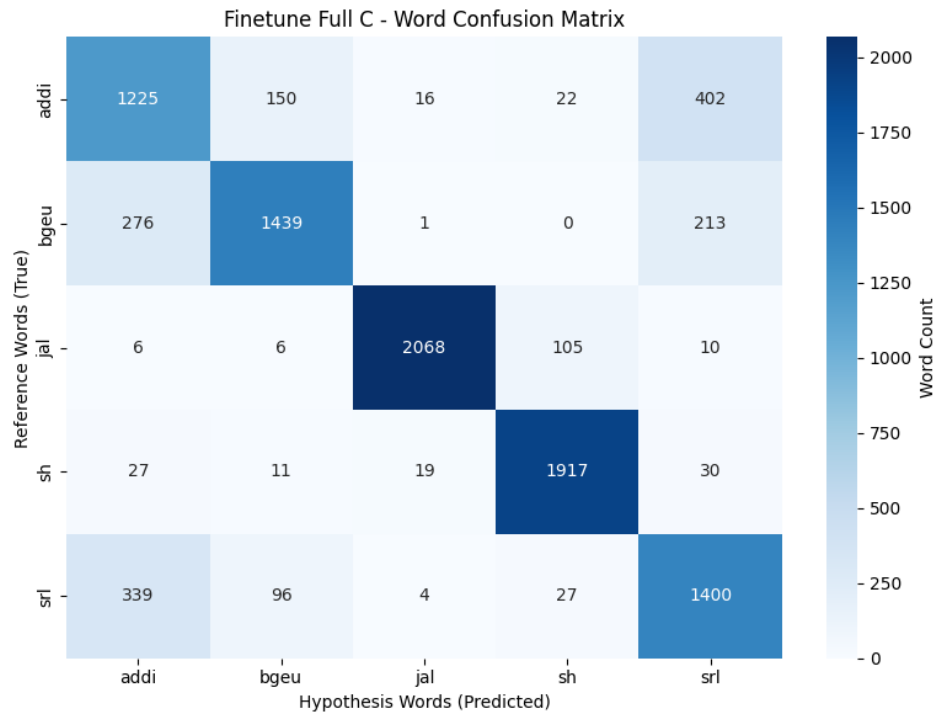


Figure 5.7: Word confusion matrix for Finetune Full C. Insertion and deletion alignment operations are not used for its creation.

The confusion analysis reveals clear patterns in model performance. Instructions like `jal` and `sh` show minimal classification errors, suggesting they produce highly distinctive power signatures. Conversely, `addi` and `srl` are frequently confused with each other, indicating similar power consumption patterns.

This error pattern corroborates our segmentation analysis findings: the models are capable of learning temporal alignment, as evidenced by low SER values (6.9-11.4%), but face challenges in discriminating between instructions with similar power characteristics. The localized nature of these confusions - concentrated among specific instruction pairs rather than distributed randomly - demonstrates that the fundamental ASR architecture is sound, with remaining errors reflecting the inherent difficulty of distinguishing subtle power signature differences rather than systematic segmentation failures.

Chapter 6

Conclusions

Our results demonstrate that ASR models, specifically wav2vec 2.0 with CTC loss, can successfully perform SCBD using only single power trace queries while automatically handling temporal segmentation. The best-performing model achieved a WER of 23.39% on the test set, corresponding to an instruction-level accuracy of 76.61%.

The substantially lower SER rates (6.9-11.4%) compared to overall WER values provide compelling evidence that ASR architectures can effectively solve the segmentation problem in SCA.

The error analysis reveals that remaining challenges are concentrated to discriminating between instructions with similar power consumption characteristics, rather than in temporal boundary detection.

Our initial WER results were achieved with a temporal resolution of only 16 samples per instruction cycle, suggesting that this discrimination task is surmountable with further optimization and/or increased temporal resolution.

6.1 Limitations

Several limitations constrain the immediate applicability of these results. Most significantly, we restricted our investigation to only five RV32I instructions, preventing practical deployment. The necessity for this constraint stems from the low oversampling rate being insufficient for discriminating between instructions of identical format that produce subtle power signature differences.

Additionally, our evaluation was conducted on a single hardware platform with controlled execution conditions. Real-world deployment would require validation across diverse processors, power delivery configurations, and

measurement environments. The models' robustness to noise, environmental variations, and countermeasures remains untested.

6.2 Future Work

Several research directions emerge from this work:

6.2.1 Immediate Extensions

Expanding to higher oversampling rates and the complete instruction set represents the most direct path to practical utility. Systematic exploration of the relationship between oversampling factor, convolutional kernel size, and stride configuration could establish domain-specific design principles. End-to-end optimization of codebook size and diversity loss weighting may further improve performance.

6.2.2 Advanced Architectures

Modern sequence modeling approaches offer promising directions. Adapting data2vec 2.0 for multimodal fusion of EM and power traces could enhance discriminative capability [43]. RNN transducers or Token-and-Duration Transducer losses may improve fine-tuning efficiency while enabling exploitation of instruction-level language modeling for reverse engineering applications requiring long-range dependency modeling [44, 45].

6.2.3 Fundamental Research Questions

With more sophisticated models, several compelling research directions emerge:

1. Can we create foundation models that work across different hardware platforms implementing the same ISA?
2. Can we train models to disassemble multiple instruction streams occurring simultaneously in different cores?
3. Can ASR models handle hardware-level optimizations like out-of-order execution, branch misses, and cache effects?
4. Can these models decode at the register, or possibly even bit level?

These directions could establish SCBD as a robust complement to traditional reverse engineering techniques, with applications spanning security analysis, intellectual property protection, and hardware validation.

References

- [1] P. Kocher, J. Jaffe, B. Jun, G. Goos, J. Van Leeuwen, and J. Hartmanis, “Differential Power Analysis,” in *Advances in Cryptology — CRYPTO’99*, M. Wiener, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 1999, vol. 1666, pp. 388–397. ISBN 978-3-540-66347-8 978-3-540-48405-9 Series Title: Lecture Notes in Computer Science. [Online]. Available: http://link.springer.com/10.1007/3-540-48405-1_25 [Page 1.]
- [2] F. Arkannezhad, P. Aghanoury, J. Feng, H. Khalili, and N. Sehatbakhsh, “SideGuard: Non-Invasive On-Chip Malware Detection in Heterogeneous IoT Systems by Leveraging Side-Channels,” in *2024 IEEE Security and Privacy Workshops (SPW)*, May 2024. doi: 10.1109/SPW63631.2024.00030 pp. 253–259, iSSN: 2770-8411. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10579519> [Page 1.]
- [3] Z. Li and D. Zhao, “ZeroD-fender: A Resource-aware IoT Malware Detection Engine via Fine-grained Side-channel Analysis,” *ACM Trans. Des. Autom. Electron. Syst.*, vol. 29, no. 6, pp. 100:1–100:25, Oct. 2024. doi: 10.1145/3687482. [Online]. Available: <https://dl.acm.org/doi/10.1145/3687482> [Page 1.]
- [4] H. A. Khan, N. Sehatbakhsh, L. N. Nguyen, M. Prvulovic, and A. Zajić, “Malware Detection in Embedded Systems Using Neural Network Model for Electromagnetic Side-Channel Signals,” *Journal of Hardware and Systems Security*, vol. 3, no. 4, pp. 305–318, Dec. 2019. doi: 10.1007/s41635-019-00074-w. [Online]. Available: <https://doi.org/10.1007/s41635-019-00074-w> [Pages 1 and 28.]
- [5] A. Sayakkara, N.-A. Le-Khac, and M. Scanlon, “Leveraging Electromagnetic Side-Channel Analysis for the Investigation of

- IoT Devices,” *Digital Investigation*, vol. 29, pp. S94–S103, Jul. 2019. doi: 10.1016/j.diin.2019.04.012. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S1742287619301616> [Page 1.]
- [6] Y. Han, M. Chan, Z. Aref, N. O. Tippenhauer, and S. Zonouz, “Hiding in Plain Sight? On the Efficacy of Power Side Channel-Based Control Flow Monitoring,” in *31st USENIX Security Symposium (USENIX Security 22)*, 2022. ISBN 978-1-939133-31-1 pp. 661–678. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/han> [Page 1.]
- [7] H. Maghrebi, T. Portigliatti, and E. Prouff, “Breaking Cryptographic Implementations Using Deep Learning Techniques,” in *Security, Privacy, and Applied Cryptography Engineering*, C. Carlet, M. A. Hasan, and V. Saraswat, Eds. Cham: Springer International Publishing, 2016, vol. 10076, pp. 3–26. ISBN 978-3-319-49444-9 978-3-319-49445-6 Series Title: Lecture Notes in Computer Science. [Online]. Available: http://link.springer.com/10.1007/978-3-319-49445-6_1 [Page 2.]
- [8] V. V. Iyer, A. Thimmaiah, M. Orshansky, A. Gerstlauer, and A. E. Yilmaz, “A Hierarchical Classification Method for High-accuracy Instruction Disassembly with Near-field EM Measurements,” *ACM Transactions on Embedded Computing Systems*, vol. 23, no. 1, pp. 1–21, Jan. 2024. doi: 10.1145/3629167 Publisher: Association for Computing Machinery (ACM). [Online]. Available: <https://dl.acm.org/doi/10.1145/3629167> [Pages 4 and 29.]
- [9] D. Krishnankutty, Z. Li, R. Robucci, N. Banerjee, and C. Patel, “Instruction Sequence Identification and Disassembly Using Power Supply Side-Channel Analysis,” *IEEE Transactions on Computers*, vol. 69, no. 11, pp. 1639–1653, Nov. 2020. doi: 10.1109/tc.2020.3018092 Publisher: Institute of Electrical and Electronics Engineers (IEEE). [Online]. Available: <https://ieeexplore.ieee.org/document/9171473/> [Pages 4, 28, and 29.]
- [10] J. Park, X. Xu, Y. Jin, D. Forte, and M. Tehranipoor, “Power-based Side-Channel Instruction-level Disassembler,” in *2018 55th ACM/ESDA/IEEE Design Automation Conference (DAC)*, Jun. 2018. doi: 10.1109/DAC.2018.8465848 pp. 1–6. [Online]. Available: <https://ieeexplore.ieee.org/document/8465848> [Pages 4, 28, and 29.]

- [11] Deloitte Touche Tohmatsu, “When Legacy is Not Enough: Understanding the High Stakes Game of Replacing a Bank’s Core Systems,” Deloitte, Tech. Rep., 2008. [Online]. Available: <https://www2.deloitte.com/content/dam/Deloitte/global/Documents/Financial-Services/gx-fsi-us-why-legacy-is-not-enough-2008.pdf> [Page 5.]
- [12] RISC-V International, “The RISC-V Instruction Set Manual Volume I: Unprivileged ISA,” Jul. 2025. [Online]. Available: <https://github.com/riscv/riscv-isa-manual/releases/tag/riscv-isa-release-a0e6b7a-2025-07-07> [Page 10.]
- [13] D. A. Patterson and J. L. Hennessy, *Computer organization and design: the hardware software interface ; RISC-V edition*. San Diego: Elsevier Science & Technology, 2018. ISBN 978-0-12-812275-4 978-0-12-812276-1 [Pages 10 and 11.]
- [14] D. M. Harris and S. L. Harris, *Digital design and computer architecture*. Amsterdam Boston: Morgan Kaufmann Publishers, 2007. ISBN 978-0-12-370497-9 [Pages 12 and 13.]
- [15] J. L. Hennessy and D. A. Patterson, *Computer architecture: a quantitative approach*, 5th ed. Amsterdam Heidelberg: Elsevier, Morgan Kaufmann, 2012. ISBN 978-0-12-383872-8 978-0-12-383873-5 [Page 13.]
- [16] F.-X. Standaert, “Introduction to Side-Channel Attacks,” in *Integrated Circuits and Systems*. Boston, MA: Springer US, 2010, pp. 27–42. ISBN 978-0-387-71827-9 978-0-387-71829-3 ISSN: 1558-9412. [Online]. Available: http://link.springer.com/10.1007/978-0-387-71829-3_2 [Page 15.]
- [17] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks,” in *Proceedings of the 23rd international conference on Machine learning*, ser. ICML ’06. New York, NY, USA: Association for Computing Machinery, Jun. 2006. doi: 10.1145/1143844.1143891. ISBN 978-1-59593-383-6 pp. 369–376. [Online]. Available: <https://dl.acm.org/doi/10.1145/1143844.1143891> [Pages 16 and 18.]

- [18] A. Hannun, “Sequence Modeling with CTC,” *Distill*, vol. 2, no. 11, p. 10.23915/distill.00008, Nov. 2017. doi: 10.23915/distill.00008. [Online]. Available: <https://distill.pub/2017/ctc> [Pages 16 and 18.]
- [19] A. Graves, A.-r. Mohamed, and G. Hinton, “Speech Recognition with Deep Recurrent Neural Networks,” Mar. 2013, arXiv:1303.5778 [cs]. [Online]. Available: <http://arxiv.org/abs/1303.5778> [Page 19.]
- [20] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention Is All You Need,” 2017, arXiv:1706.03762 [cs]. [Online]. Available: <http://arxiv.org/abs/1706.03762> [Pages 19, 20, and 22.]
- [21] F. Love, “NNTikZ - TikZ Diagrams for Deep Learning and Neural Networks,” 2024, gitHub repository. [Online]. Available: <https://github.com/fraserlove/nntikz> [Pages 20, 21, and 23.]
- [22] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, “wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations,” Oct. 2020, arXiv:2006.11477 [cs]. [Online]. Available: <http://arxiv.org/abs/2006.11477> [Pages 24 and 25.]
- [23] T. Eisenbarth, C. Paar, and B. Weghenkel, “Building a Side Channel Based Disassembler,” in *Transactions on Computational Science X: Special Issue on Security in Computing, Part I*, M. L. Gavrilova, C. J. K. Tan, and E. D. Moreno, Eds. Berlin, Heidelberg: Springer, 2010, pp. 78–99. ISBN 9783642174995. [Online]. Available: https://doi.org/10.1007/978-3-642-17499-5_4 [Pages 28 and 29.]
- [24] V. Cristiani, M. Lecomte, and T. Hiscock, “A Bit-Level Approach to Side Channel Based Disassembling,” in *Smart Card Research and Advanced Applications*, S. Belaïd and T. Güneysu, Eds. Cham: Springer International Publishing, 2020, vol. 11833, pp. 143–158. ISBN 978-3-030-42067-3 978-3-030-42068-0 Series Title: Lecture Notes in Computer Science. [Online]. Available: http://link.springer.com/10.1007/978-3-030-42068-0_9 [Pages 28 and 29.]
- [25] J. Maillard, T. Hiscock, M. Lecomte, and C. Clavier, “Towards Fine-grained Side-Channel Instruction Disassembly on a System-on-Chip,” in *2022 25th Euromicro Conference on Digital System Design (DSD)*, Aug. 2022. doi: 10.1109/DSD57027.2022.00069 pp. 472–479, iSSN:

- 2771-2508. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9996957> [Page 28.]
- [26] J. Van Geest and I. Buhan, “A Side-Channel Based Disassembler for the ARM-Cortex M0,” in *Lecture Notes in Computer Science*. Cham: Springer International Publishing, 2022, pp. 183–199. ISBN 978-3-031-16814-7 978-3-031-16815-4 ISSN: 0302-9743, 1611-3349. [Online]. Available: https://link.springer.com/10.1007/978-3-031-16815-4_11 [Pages 28 and 29.]
- [27] H. Fendri, M. Macchetti, J. Perrine, and M. Stojilović, “A Deep-Learning Approach to Side-Channel Based CPU Disassembly at Design Time,” in *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, Mar. 2022. doi: 10.23919/DATE54114.2022.9774531 pp. 670–675, iSSN: 1558-1101. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9774531> [Pages 28 and 29.]
- [28] S. Alabdulwahab, M. Cheong, A. Seo, Y.-T. Kim, and Y. Son, “Enhancing deep learning-based side-channel analysis using feature engineering in a fully simulated IoT system,” *Expert Systems with Applications*, vol. 266, p. 126079, Mar. 2025. doi: 10.1016/j.eswa.2024.126079 Publisher: Elsevier BV. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0957417424029464> [Page 28.]
- [29] S. Yang, Y. Zhou, J. Liu, and D. Chen, “Back Propagation Neural Network Based Leakage Characterization for Practical Security Analysis of Cryptographic Implementations,” in *Information Security and Cryptology - ICISC 2011*, D. Hutchison, T. Kanade, J. Kittler, J. M. Kleinberg, F. Mattern, J. C. Mitchell, M. Naor, O. Nierstrasz, C. Pandu Rangan, B. Steffen, M. Sudan, D. Terzopoulos, D. Tygar, M. Y. Vardi, G. Weikum, and H. Kim, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, vol. 7259, pp. 169–185. ISBN 978-3-642-31911-2 978-3-642-31912-9 Series Title: Lecture Notes in Computer Science. [Online]. Available: http://link.springer.com/10.1007/978-3-642-31912-9_12 [Page 28.]
- [30] H. Li, M. Ninan, B. Wang, and J. M. Emmert, “TinyPower: Side-Channel Attacks with Tiny Neural Networks,” in *2024 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. Tysons Corner, VA, USA: IEEE, May 2024. doi:

- 10.1109/host55342.2024.10545382 pp. 320–331. [Online]. Available: <https://ieeexplore.ieee.org/document/10545382/> [Page 28.]
- [31] D. Asonov and R. Agrawal, “Keyboard acoustic emanations,” in *IEEE Symposium on Security and Privacy, 2004. Proceedings. 2004.* Berkeley, CA, USA: IEEE, 2004. doi: 10.1109/secpri.2004.1301311 pp. 3–11. [Online]. Available: <http://ieeexplore.ieee.org/document/1301311/> [Page 29.]
- [32] L. Zhuang, F. Zhou, and J. D. Tygar, “Keyboard acoustic emanations revisited,” *ACM Transactions on Information and System Security*, vol. 13, no. 1, pp. 1–26, Oct. 2009. doi: 10.1145/1609956.1609959 Publisher: Association for Computing Machinery (ACM). [Online]. Available: <https://dl.acm.org/doi/10.1145/1609956.1609959> [Page 29.]
- [33] lowRISC, “Ibex Demo System.” [Online]. Available: <https://github.com/lowRISC/ibex-demo-system> [Pages 32 and 33.]
- [34] —, “Ibex RISC-V Core.” [Online]. Available: <https://github.com/lowRISC/ibex> [Page 33.]
- [35] —, “Ibex Documentation.” [Online]. Available: <https://ibex-core.readthedocs.io/en/latest/> [Page 33.]
- [36] Xilinx, “7 Series FPGAs Clocking Resources - User Guide.” [Online]. Available: https://docs.amd.com/v/u/en-US/ug472_7Series_Clocking [Page 34.]
- [37] NewAE, “ChipWhisperer.” [Online]. Available: <https://github.com/newaetech/chipwhisperer> [Page 34.]
- [38] lowRISC, “Spike RISC-V ISA Simulator.” [Online]. Available: <https://github.com/lowRISC/riscv-isa-sim> [Page 37.]
- [39] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. Von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest, and A. Rush, “Transformers: State-of-the-Art Natural Language Processing,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Online: Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.emnlp-demos.6. [Online]. Available: <https://www.aclweb.org/anthology/2020.emnlp-demos.6> [Page 43.]

- [40] T. Akiba, S. Sano, T. Yanese, T. Ohta, and M. Koyama, “Optuna: A Next-generation Hyperparameter Optimization Framework,” in *Proceedings of the 25th {ACM} {SIGKDD} International Conference on Knowledge Discovery and Data Mining*, 2019. [Pages 44 and 45.]
- [41] Optuna, “Optuna: A hyperparameter optimization framework.” [Online]. Available: <https://github.com/optuna/optuna> [Page 45.]
- [42] D. Jurafsky and J. H. Martin, *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*, 3rd ed. Self-published, 2025, online manuscript released January 12, 2025. [Online]. Available: <https://web.stanford.edu/~jurafsky/slp3/> [Pages 49, 50, and 79.]
- [43] A. Baevski, A. Babu, W.-N. Hsu, and M. Auli, “Efficient self-supervised learning with contextualized target representations for vision, speech and language,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML’23, vol. 202. Honolulu, Hawaii, USA: JMLR.org, Jul. 2023, pp. 1416–1429. [Page 64.]
- [44] A. Graves, “Sequence Transduction with Recurrent Neural Networks,” Nov. 2012, arXiv:1211.3711 [cs]. [Online]. Available: <http://arxiv.org/abs/1211.3711> [Page 64.]
- [45] H. Xu, F. Jia, S. Majumdar, H. Huang, S. Watanabe, and B. Ginsburg, “Efficient sequence transduction by jointly predicting tokens and durations,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML’23, vol. 202. Honolulu, Hawaii, USA: JMLR.org, Jul. 2023, pp. 38462–38484. [Page 64.]
- [46] Q. McNemar, “Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, Jun. 1947. doi: 10.1007/bf02295996 Publisher: Cambridge University Press (CUP). [Online]. Available: https://www.cambridge.org/core/product/identifier/S0033312300045178/type/journal_article [Page 78.]
- [47] A. L. Edwards, “Note on the “Correction for Continuity” in Testing the Significance of the Difference between Correlated Proportions,” *Psychometrika*, vol. 13, no. 3, pp. 185–187, Sep. 1948. doi: 10.1007/bf02289261 Publisher: Cambridge University Press (CUP).

[Online]. Available: https://www.cambridge.org/core/product/identifier/S0033312300045452/type/journal_article [Page 78.]

Appendix A

Trace Visualization

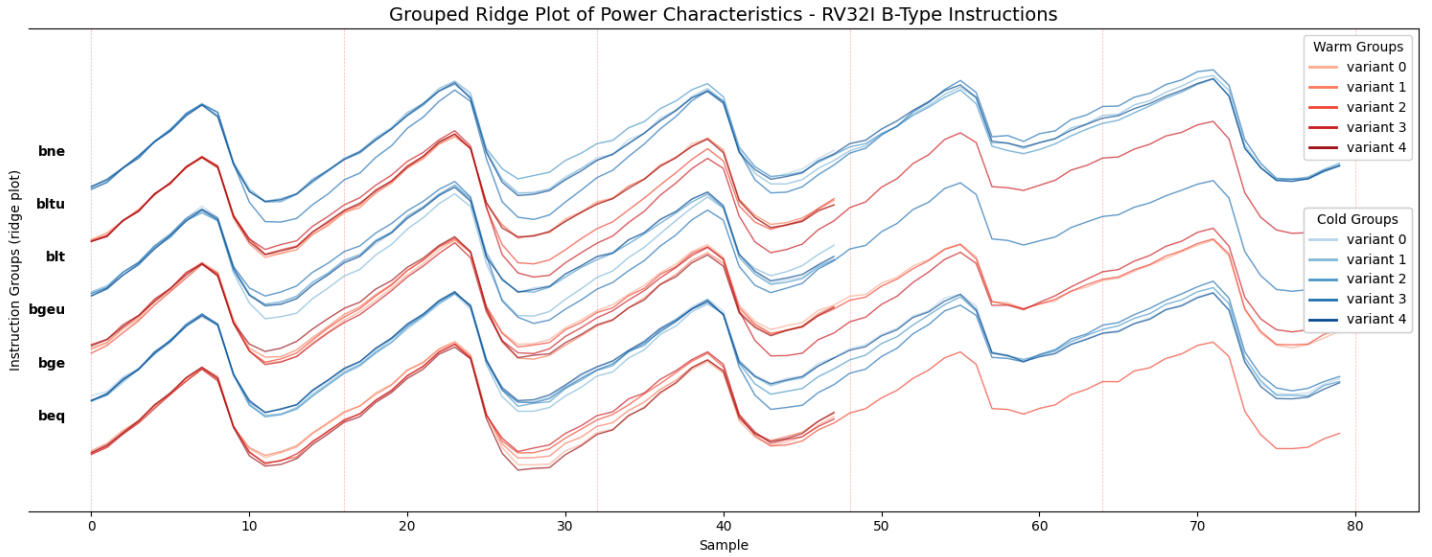


Figure A.1: Ridge plot of power traces for RV32I B-Type instructions.

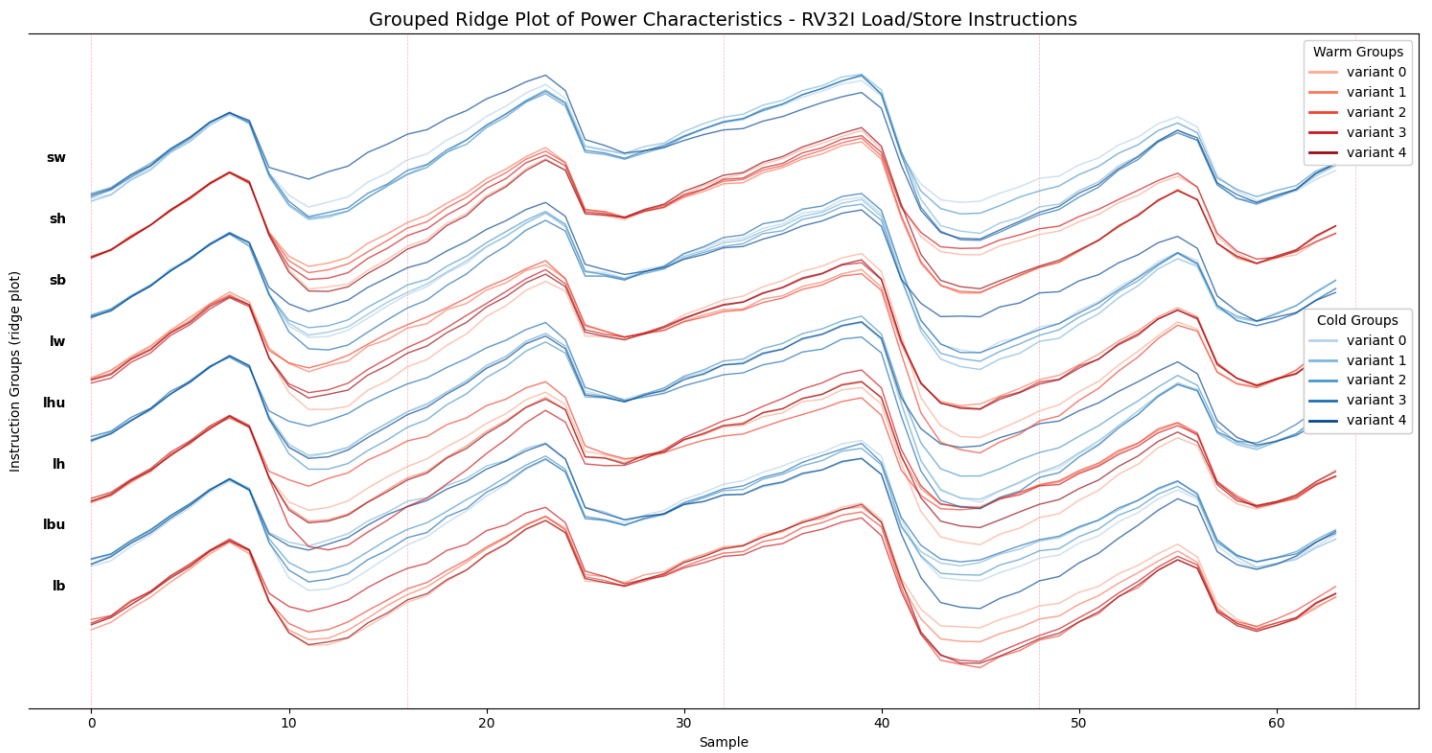


Figure A.2: Ridge plot of power traces for RV32I Load/Store-Type instructions.

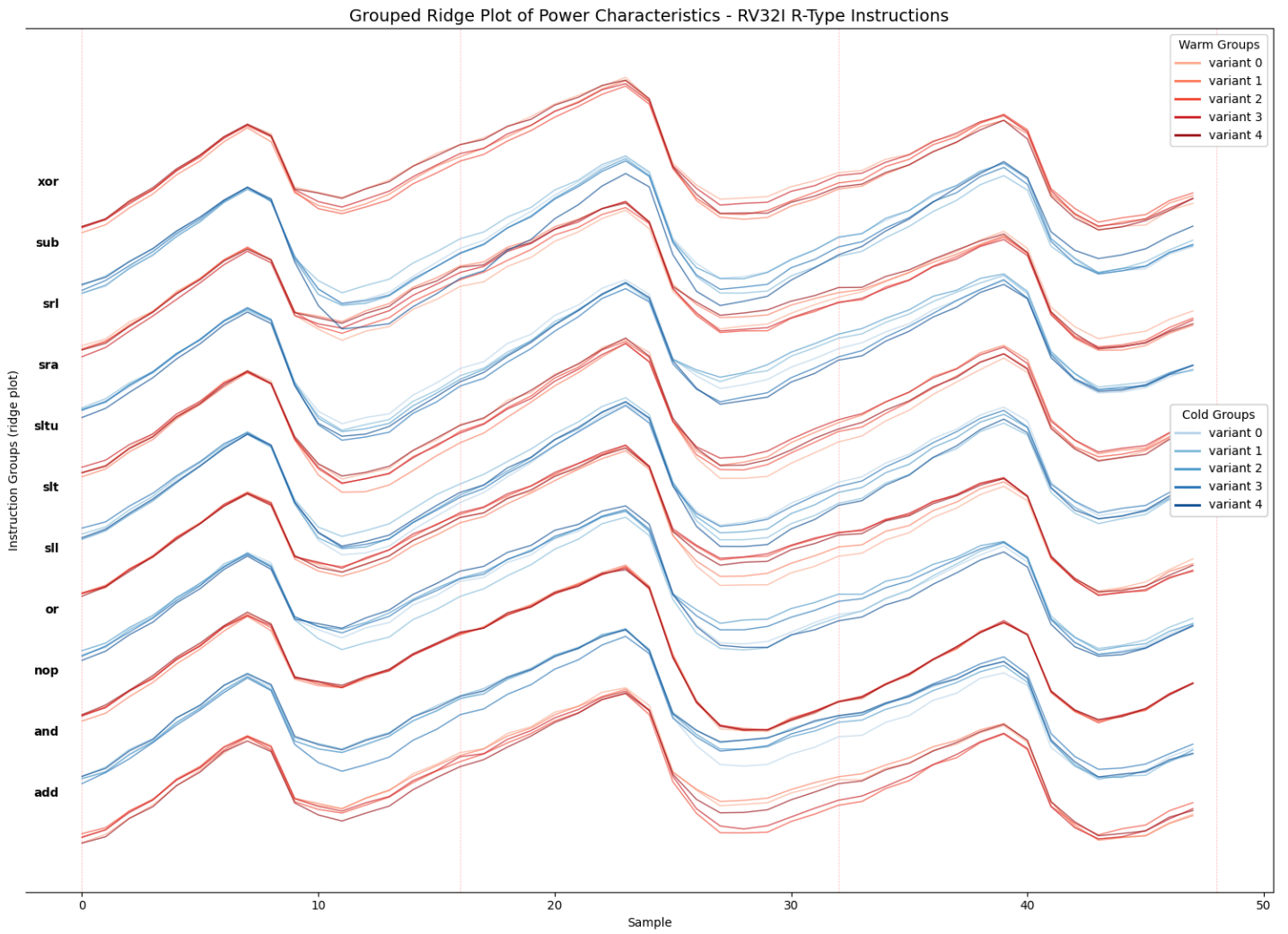


Figure A.3: Ridge plot of power traces for RV32I R-Type instructions.

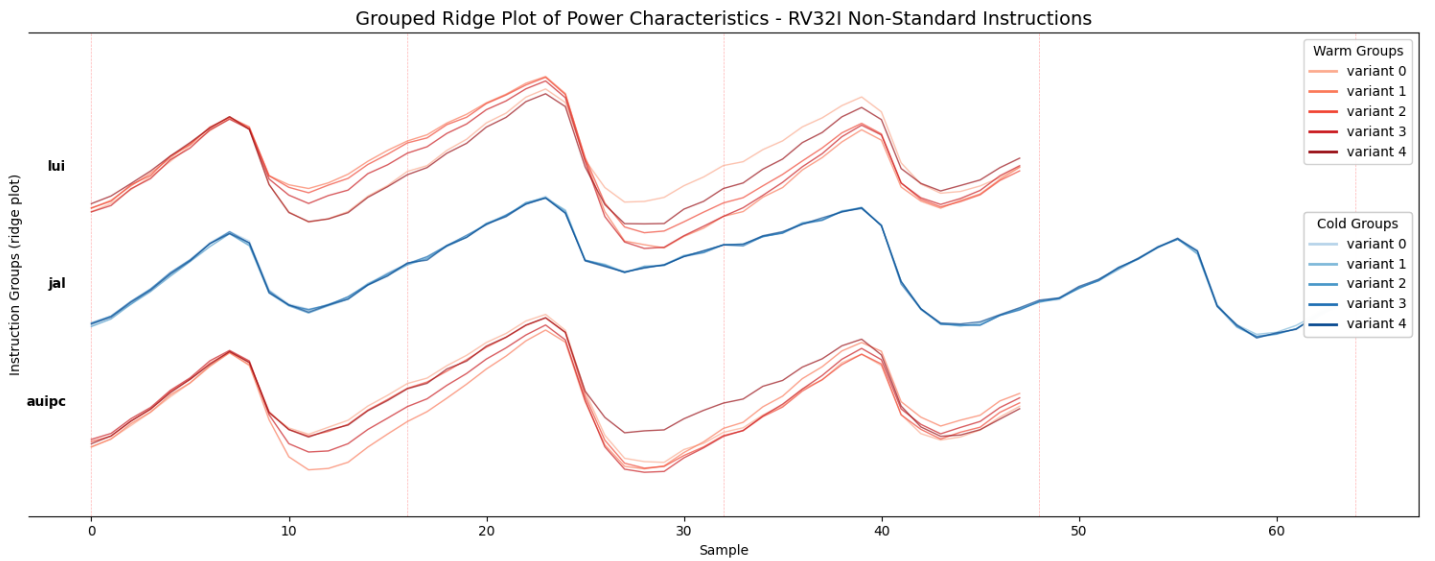


Figure A.4: Ridge plot of power traces for RV32I Non-Standard instructions.

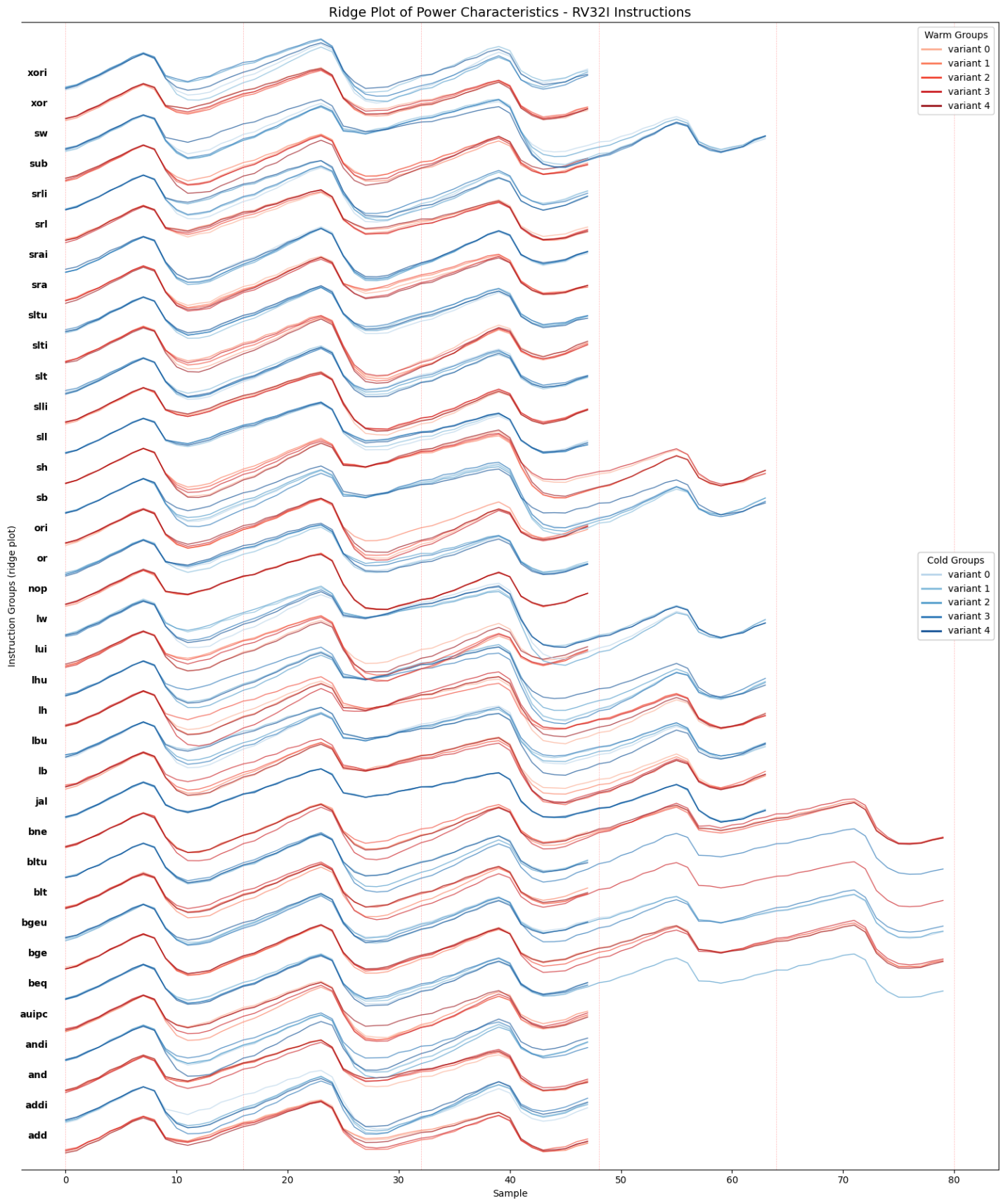


Figure A.5: Ridge plot of power traces for all RV32I instructions.

Appendix B

ASR Adapted McNemar Derivation

This appendix provides the detailed mathematical derivation of the ASR adapted McNemar test introduced in Section 4.5.3.

B.1 Standard McNemar Test Foundation

The McNemar test compares two classifiers by examining whether there is a systematic difference in their disagreements [46]. The test uses a contingency table (Table B.1) where disagreements are represented by the discordant pairs labeled b and c .

H_a (Our Model)	H_0 (Baseline)	
	Correct	Incorrect
Correct	a	b
Incorrect	c	d

Table B.1: Contingency table for correctness of two classifiers, H_0 and H_a .

We employ the continuity corrected McNemar’s test, which is more conservative and has become the default choice for most applications [47]. The test statistic is:

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c}. \quad (\text{B.1})$$

B.2 Adaptation for Transcription Models

Our transcription models differ from standard classifiers in that they can insert or delete words in addition to making correct/incorrect transcriptions. We represent the performance of each model H_i by partitioning each word according to its edit operation:

$$Ops_i = C_i \cup S_i \cup D_i \cup I_i,$$

where C_i , S_i , D_i , and I_i denote the sets of words belonging to edit operations Correct, Substitution, Deletion, and Insertion under model H_i , respectively. These sets are mutually disjoint.

For our baseline model H_0 (naive majority class), we define $I_0 = D_0 = \emptyset$, as this baseline can segment correctly and only makes substitution errors.

B.3 Constructing Discordant Pairs

The discordant pairs can be expressed in terms of these edit operation sets:

$$b = |C_a \cap S_0|$$

$$c = |S_a \cap C_0| + |D_a \cap C_0| = |(S_a \cup D_a) \cap C_0|$$

where the second equality follows from the mutual disjointness of S_a and D_a .

B.4 Assignment of Insertions

The challenge lies in assigning the insertion errors I_a . To ensure a stringent evaluation, we assign all insertions to discordant pair c , which reduces the test statistic and makes it harder to demonstrate statistical significance. This assignment treats insertions as cases where the baseline model was correct but our model was incorrect.

Let us verify the aforementioned conservative bias. We have that for all $b, c \in \mathbb{N}_0$ such that $b > c + |I_a|$, it follows that

$$(|b - (c + |I_a|)| - 1)^2 \leq (|b - c| - 1)^2$$

This inequality demonstrates that adding the insertion count to discordant pair c reduces the test statistic (Equation B.1), by reducing the numerator and increasing the denominator, thus biasing the test against our model - provided the assumption $b > c + |I_a|$ holds, i.e. our model performs reasonably well.

B.5 Final Test Statistic

Incorporating the assignment of insertions, our final test statistic becomes:

$$\chi^2 = \frac{(|(S_a \cup D_a) \cap C_0| - |C_a \cap S_0| - |I_a| - 1)^2}{|(S_a \cup D_a) \cap C_0| + |C_a \cap S_0| + |I_a|}$$

This formulation is valid under the assumption:

$$|(S_a \cup D_a) \cap C_0| + |I_a| < |C_a \cap S_0|.$$

B.6 Independence Assumption

The validity of McNemar's test requires that errors be independent. While this assumption is often violated in speech recognition due to contextual dependencies between neighboring words [42, ch. 16.5], our domain of independent instruction sequences with associated power traces makes this assumption more reasonable, though not guaranteed to hold in all cases.

Appendix C

Hyperparameter Tuning Visuals

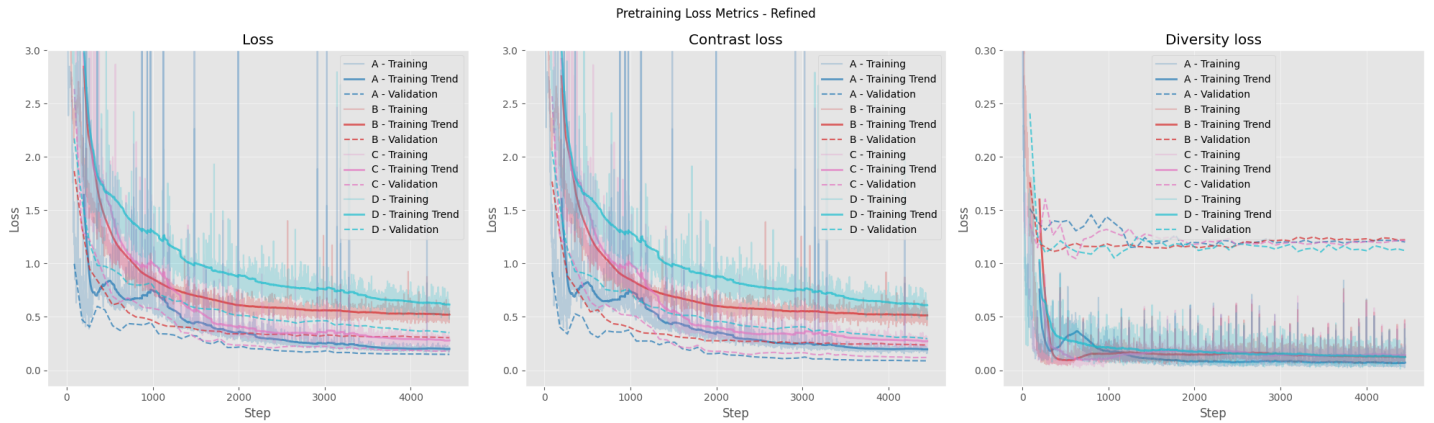


Figure C.1: Metrics for the pretrain-refined-[] stage. Both loss, loss trendline and validation is shown for each branch in each loss category.

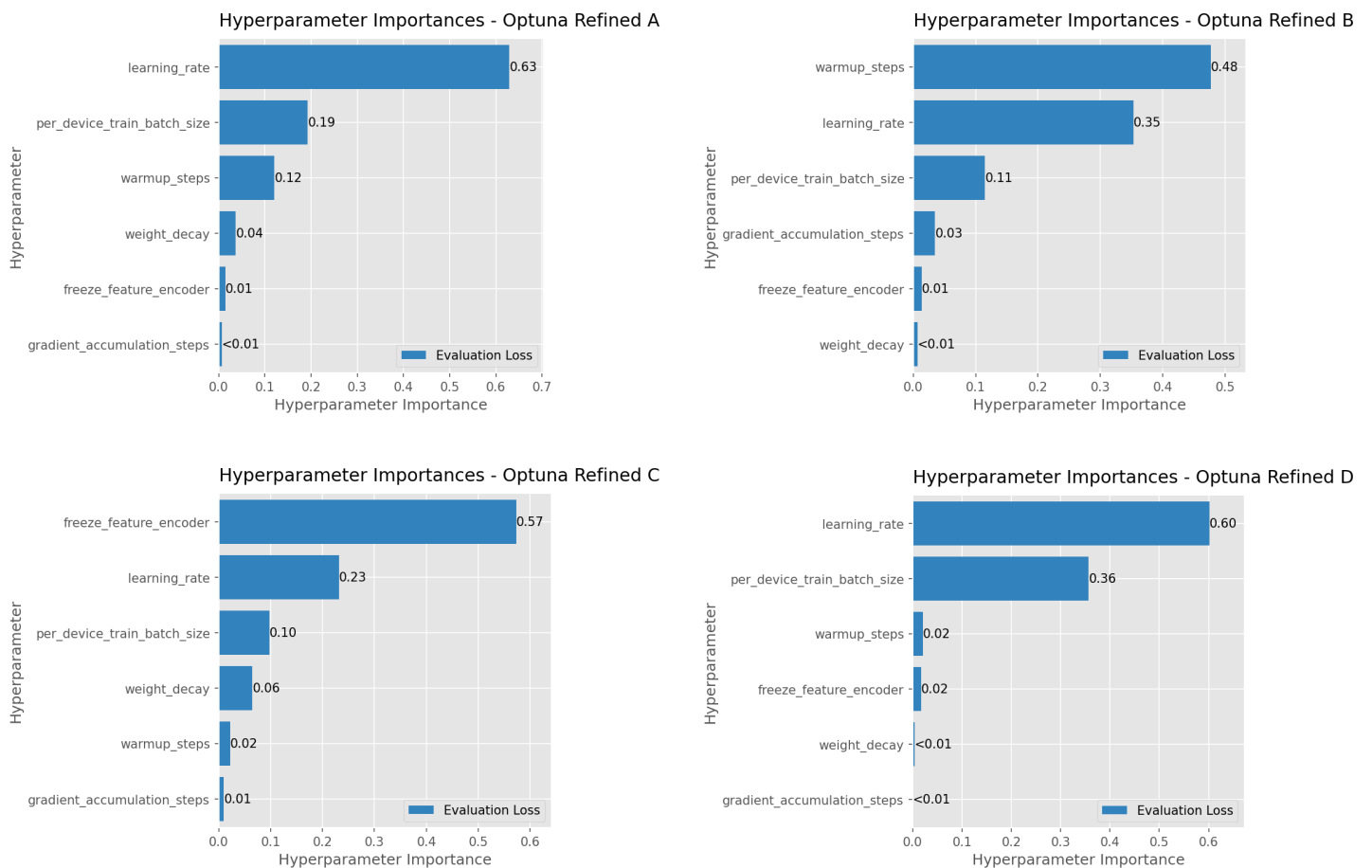


Figure C.2: Finetuning stage Optuna parameter importances for branches A–D.

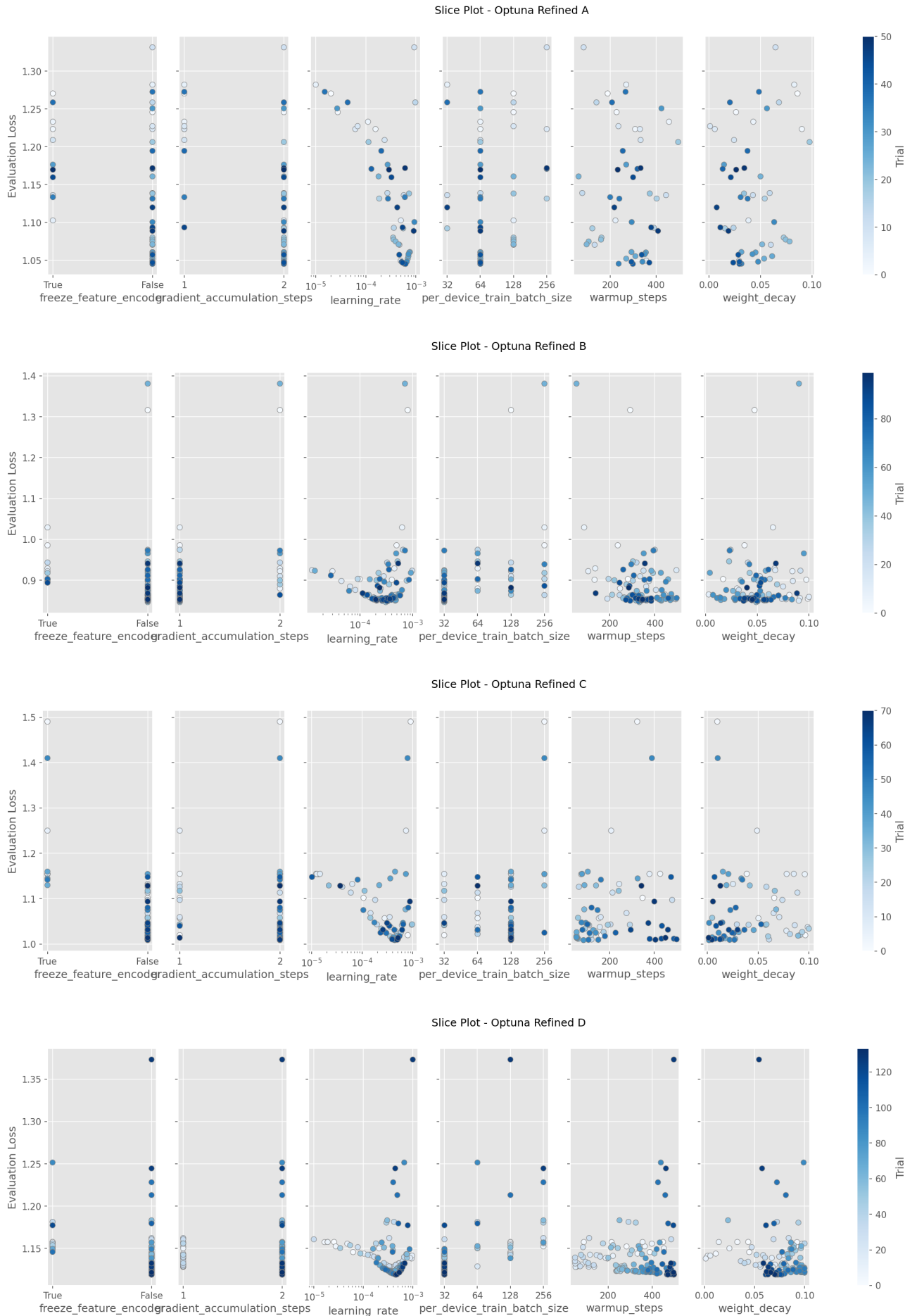


Figure C.3: Finetuning stage Optuna parameter importances for branches A–D.

Appendix D

Word Confusion Matrices

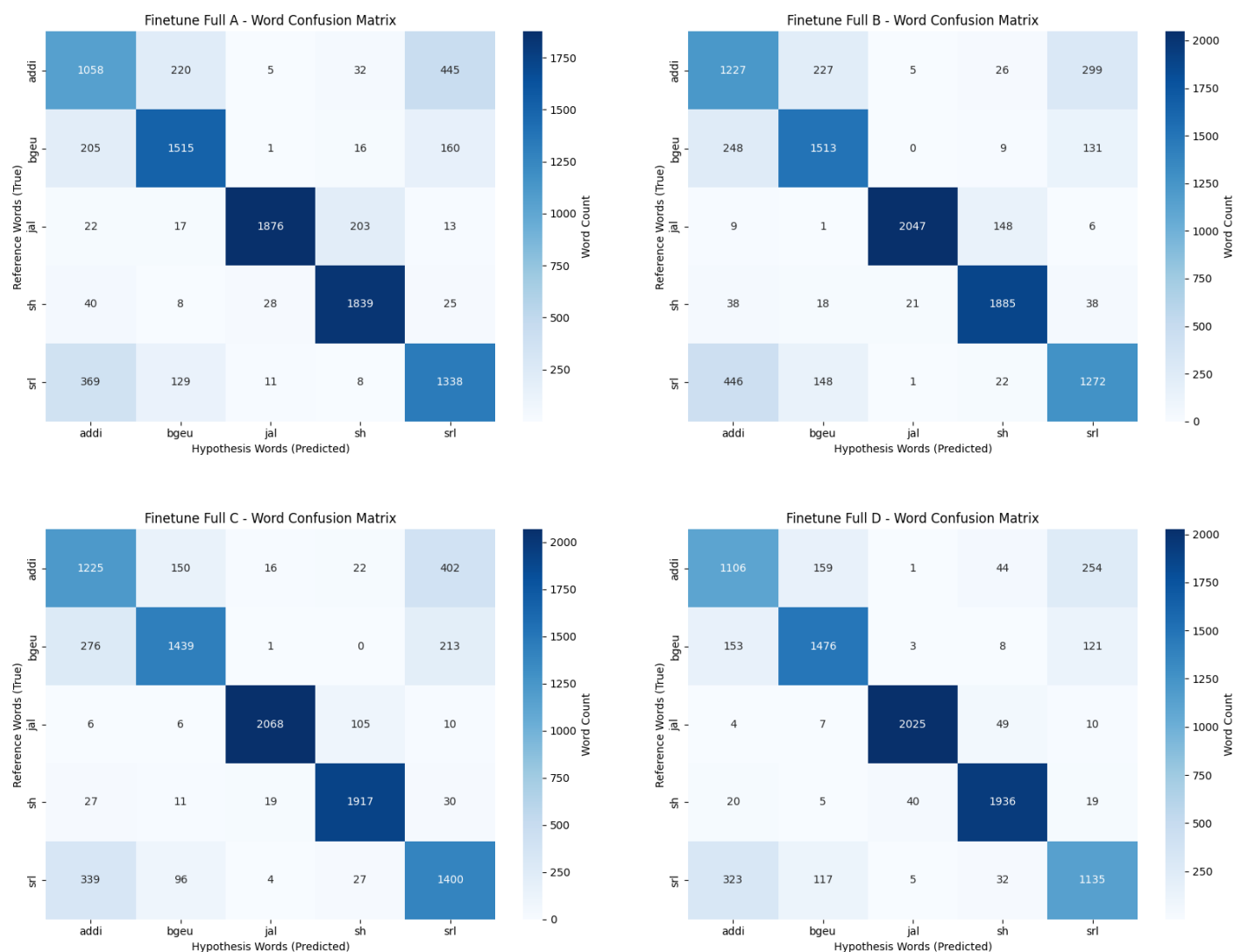


Figure D.1: Word confusion matrices for each of the Finetune Full A-D models evaluated on the test split of the full_rvi_mdb dataset. Insertion and delection operations from alignment are ignored.

€€€€ For DIVA €€€€

```
{
  "Author1": { "Last name": "Duedahl",
    "First name": "Mathias",
    "Local User Id": "u13ajbkk",
    "E-mail": "duedahl@kth.se",
    "organisation": { "L1": "School of Electrical Engineering and Computer Science",
      }
    },
  "Cycle": "2",
  "Course code": "DA246X",
  "Credits": "30.0",
  "Degree1": { "Educational program": "Master's Programme, Computer Science, 120 credits"
    , "programcode": "TCSCM"
    , "Degree": "Master's Programme, Computer Science, 120 credits"
    , "subjectArea": "Communication Systems"
    },
  "Title": {
    "Main title": "Single-Query Auto-Segmenting Side-Channel Based Disassembly",
    "Language": "eng"
    },
  "Alternative title": {
    "Main title": "Enmätnings Auto-Segmenterande Sidokanalsbaserad Instruktionsavkodning",
    "Language": "swe"
    },
  "Supervisor1": { "Last name": "Sternby",
    "First name": "Jakob",
    "E-mail": "jakob.sternby@ericsson.com",
    "Other organisation": "Ericsson Research"
    },
  "Supervisor2": { "Last name": "Jendral",
    "First name": "Sönke",
    "Local User Id": "u15sve5b",
    "E-mail": "jendral@kth.se",
    "organisation": { "L1": "School of Electrical Engineering and Computer Science",
      "L2": "Division of Electronics and Embedded Systems"
    }
    },
  "Examiner1": { "Last name": "Dubrova",
    "First name": "Elena",
    "Local User Id": "u14uxzhv",
    "E-mail": "dubrova@kth.se",
    "organisation": { "L1": "School of Electrical Engineering and Computer Science",
      "L2": "Division of Electronics and Embedded Systems"
    }
    },
  "Cooperation": { "Partner_name": "Ericsson",
    "National Subject Categories": "10201, 20203",
    "Other information": { "Year": "2025", "Number of pages": "xvii,83" },
    "Copyrightleft": "copyright",
    "Series": { "Title of series": "TRITA – EECS-EX", "No. in series": "2024:0000" },
    "Opponents": { "Name": "C. Perdakis",
      "Presentation": { "Date": "2025-08-19 11:00"
        , "Language": "eng"
        , "Room": "via Zoom https://kth-se.zoom.us/j/6377551384"
        , "Address": "TBD"
        , "City": "Stockholm"
        },
      "Number of lang instances": "2",
      "Abstract[eng]": "€€€€"
    }
  }
```

\gls{SCA} has long been a cornerstone of cryptographic security research, enabling attackers to extract sensitive information by monitoring a device's power consumption or \gls{EM} emissions. This foundation has led researchers to explore \gls{SCBD}, which aims to reconstruct a program's instruction sequence directly from side-channel traces. However, practically viable solutions remain elusive due to the trace segmentation bottleneck: existing techniques either rely on averaged traces, classify only isolated instructions, or require custom, device-specific logic to segment continuous traces.

This segmentation challenge has been extensively studied in \gls{ASR}, where segmenting continuous audio streams into discrete phonemes or words is critical. Despite striking similarities between \gls{SCA} and \gls{ASR} challenges, cross-domain approaches remain unexplored in \gls{SCBD}. We investigate whether modern \gls{ASR} models can be adapted to solve the segmentation problem in \gls{SCBD}.

We developed a comprehensive experimental framework using a custom \gls{RV} \gls{ISG} and implemented a \gls{RV}32 softcore on a \gls{FPGA} to generate training data. We adapted the wav2vec-2.0 architecture, pretraining on unlabeled power trace data followed by fine-tuning with \gls{CTC} loss on labeled instruction sequences. This approach automatically handles segmentation while learning instruction-specific power signatures.

Our method achieved an accuracy of 76.61\% for sequences containing five different \gls{RV}32I instructions, with \gls{SER} as low as 6.86\%. These results demonstrate that \gls{ASR} models can effectively solve the temporal alignment challenge in \gls{SCBD}, with remaining errors concentrated in instruction discrimination rather than segmentation. Statistical validation confirms significant improvements over baseline approaches ($p < 0.001$). Our findings establish \gls{ASR} as a viable foundation for \gls{SCBD} and demonstrate the potential of cross-domain approaches for practical disassembly techniques.

€€€€,
"Keywords[eng]": €€€€
Side-Channel Analysis, Side-Channel Based Disassembly, Automatic Speech Recognition, wav2vec 2.0, Connectionist Temporal Classification €€€€,
"Abstract[swe]": €€€€

Sidokanalsanalys (SCA) har länge varit en hörnsten inom kryptografisk säkerhetsforskning, och möjliggör för angripare att extrahera känslig information genom att övervaka en enhets strömförbrukning eller elektromagnetiska (EM) läckage. Denna grund har lett forskare att utforska sidokanalbaserad instruktionsavkodning (SCBD), som syftar till att rekonstruera ett programs instruktionssekvens direkt från sidokanalsmätningar. Dock förblir praktiskt användbara lösningar orealistiska på grund av flaskhalsen för mätning-segmentering: befintliga tekniker förlitar sig antingen på medelvärdesbildning, klassificerar endast isolerade instruktioner, eller kräver anpassad, enhetsspecifik logik för att segmentera kontinuerliga mätningar.

Denna segmenteringsutmaning har studerats utförligt inom automatisk taligenkänning (ASR), där segmentering av kontinuerliga ljudströmmar till diskreta fonem eller ord är kritisk. Trots slående likheter i problembeskrivningen för SCA och ASR har ansatser från taligenkänningsområdet förblivit outforskade inom SCBD. Vi undersöker huruvida moderna ASR-modeller kan anpassas för att lösa segmenteringsproblemet inom SCBD.

Vi har utvecklat ett omfattande experimentellt ramverk med hjälp av en anpassad RISC-V (RV) instruktionssekvensgenererare (ISG) och implementerade en RV32 softcore på en fältprogrammerbar grindmatris (FPGA) för att generera träningsdata. Vi anpassade wav2vec 2.0-arkitekturen, förtränade på omärkta strömspårsdata följt av finjustering med konnektionistisk temporal klassificering (CTC)-förlust på märkta instruktionssekvenser. Denna ansats hanterar automatiskt segmentering samtidigt som den lär sig instruktionsspecifika strömsignaturer.

Vår metod uppnådde en noggrannhet på 76.61\% för sekvenser innehållande fem olika RV32I-instruktioner, med segmenteringsfelsfrekvenser (SER) så låga som 6,86\%. Dessa resultat visar att ASR-modeller effektivt kan lösa den temporala anpassningsutmaningen inom SCBD, med kvarvarande fel koncentrerade till instruktionsdiskriminering snarare än segmentering. Statistisk validering bekräftar betydande förbättringar jämfört med en naiv jämförelsemetod ($p < 0,001$). Våra fynd etablerar ASR som en möjlig utgångspunkt för SCBD och demonstrerar potentialen av att dra lärdom från andra domäner för att göra framsteg inom metoder för instruktionsavkodning.

€€€€,
"Keywords[swe]": €€€€
Sidokanalsanalys, Sidokanalbaserad Instruktionsavkodning, Automatisk Taligenkänning, wav2vec 2.0, Konnektionistisk Temporal Klassificering €€€€,
}

acronyms.tex

```
%%% Local Variables:
%%% mode: latex
%%% TeX-master: t
%%% End:
% The following command is used with glossaries-extra
\setabbreviationstyle[acronym]{long-short}
% The form of the entries in this file is \newacronym{label}{acronym}{phrase}
%                                     or \newacronym[options]{label}{acronym}{phrase}
% see "User Manual for glossaries.sty" for the details about the options, one example is shown
% below
% note the specification of the long form plural in the line below
%\newacronym[longplural={Debugging Information Entities}]{DIE}{DIE}{Debugging Information Entity}
%
% The following example also uses options
%\newacronym[shortplural={OSes}, firstplural={operating systems (OSes)}]{OS}{OS}{operating
% system}

% example of putting in a trademark on first expansion
%\newacronym[first={NVIDIA OpenSHMEM Library (NVSHMEM\texttrademark)}]{NVSHMEM}{NVSHMEM}{NVIDIA
% OpenSHMEM Library}

##### MINE #####
% Hardware
% note the use of a non-breaking dash in long text for the following acronym
\newacronym{FPGA}{FPGA}{-fieldprogrammable gate array}
\newacronym{RV}{RV}{-RISCv}
\newacronym{SUT}{SUT}{system under test}
\newacronym{DUT}{DUT}{device under test}
\newacronym{EM}{EM}{electromagnetic}
\newacronym{RISC}{RISC}{reduced instruction set computer}
\newacronym{CISC}{CISC}{complex instruction set computer}
\newacronym[shortplural={ISAs}, firstplural={instruction set architectures
% (ISAs)}]{ISA}{ISA}{instruction set architecture}
\newacronym[shortplural={CPUs}, firstplural={central processing units (CPUs)}]{CPU}{CPU}{central
% processing unit}
\newacronym{SOC}{SoC}{system on a chip}
\newacronym{ALU}{ALU}{arithmetic logic unit}
\newacronym{PC}{PC}{program counter}
\newacronym{IR}{IR}{instruction register}
\newacronym{IF}{IF}{instruction fetch cycle}
\newacronym{ID}{ID}{instruction decode cycle}
\newacronym{EX}{EX}{execution cycle}
\newacronym{MEM}{MEM}{memory access cycle}
\newacronym{WB}{WB}{write-back cycle}
\newacronym{ISG}{ISG}{instruction sequence generator}

% Machine Learning
\newacronym{SCA}{SCA}{-sidechannel analysis}
\newacronym{SCBD}{SCBD}{-sidechannel based disassembly}
\newacronym{WER}{WER}{word error rate}
\newacronym[shortplural={SERs}, firstplural={segmentation error rates
% (SERs)}]{SER}{SER}{segmentation error rate}
\newacronym{ASR}{ASR}{automatic speech recognition}
\newacronym{CTC}{CTC}{connectionist temporal classification}
\newacronym{DNN}{DNN}{deep neural network}
\newacronym{CNN}{CNN}{convolutional neural network}
\newacronym[shortplural={RNNs}, firstplural={recurrent neural networks
% (RNNs)}]{RNN}{RNN}{recurrent neural network}
\newacronym{MLP}{MLP}{multilayer perceptron}
```

lib/acronyms.tex